



به نام خدا



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

مدل‌های مولد عمیق

تمرین چهارم درس

نام و نام خانوادگی	محمد طاهّا مجلسی کویایی
شماره دانشجویی	۸۱۰۱۵۰۴
تاریخ ارسال گزارش	۱۴۰۳/۱۰/۱۴

فهرست

مدل زبان بصری (Vision-Language Model)

۴.....	1.1. بخش اول – مدل PaliGemma
۴.....	1.1.1. زیربخش اول – مدل VLM
11.....	1.1.2. زیربخش دوم – رمزگذار تصویر SigLIP
۲۰.....	1.1.3. زیربخش سوم – پیش‌آموزش Pre-training
۲۹.....	1.1.4. زیربخش چهارم – انتقال یادگیری Transfer Learning
۴۹	1.1.5p. زیربخش پنجم – پیاده‌سازی و آموزش مدل
۸۰	هم‌خوانی جریان (Flow Matching) – سوال ۲
۸۰	زیر بخش اول
۸۴	زیر بخش دوم
۸۸	زیر بخش سوم
۹۳	مراجع

بخش اول مدل PaliGemma :

مدل **Paligemma** یک مدل پیشرفته در حوزه مدل‌های بینایی-زبانی (Vision-Language Models) است که با هدف ترکیب و یکپارچه‌سازی داده‌های بصری و متنی برای انجام وظایف پیچیده‌ای مانند پاسخ به سوالات بصری (VQA) و توصیف تصاویر طراحی شده است. این مدل با بهره‌گیری از معماری مقیاس‌پذیر خود، قادر به پردازش و

درک همزمان اطلاعات تصویری و زبانی می‌باشد که آن را به ابزاری قدرتمند در کاربردهای متنوع هوش مصنوعی تبدیل کرده است.

معماری مدل Paligemma شامل دو جزء اصلی است: **رمزگذار تصویری SigLIP** و **رمزگشا زبانی**. رمزگذار SigLIP با استفاده از تکنیک‌های پیشرفته مانند استفاده از توابع سیگموئید برای پیش‌تربیت، ویژگی‌های کلیدی تصاویر را استخراج کرده و آنها را به فضای تعبیه مشترکی منتقل می‌کند که با داده‌های متنی همسو است. این امر امکان تعامل موثر بین داده‌های بصری و زبانی را فراهم می‌آورد و به مدل اجازه می‌دهد تا به صورت هماهنگ و دقیق به سوالات مرتبط با تصاویر پاسخ دهد. از سوی دیگر، رمزگشا زبانی با استفاده از معماری ترنسفورمر، توانایی تولید متن‌های مرتبط و دقیق را بر اساس ویژگی‌های استخراج شده از تصاویر دارد.

یکی از ویژگی‌های برجسته مدل Paligemma، **تنظیم دقیق با بهره‌وری پارامتری (PEFT)** است که از روش‌هایی مانند LoRA و QLoRA برای بهینه‌سازی و کاهش نیازمندی‌های محاسباتی استفاده می‌کند. این روش‌ها به مدل اجازه می‌دهند تا با تغییرات کم در پارامترها، به سرعت و با کارایی بالا به وظایف خاصی مانند پاسخ به سوالات بصری تنظیم شود. علاوه بر این، Paligemma با استفاده از تکنیک‌های **تطابق جریان (Flow Matching)** توانسته است فرآیند تولید داده‌های جدید را بهینه کرده و قابلیت‌های مولد خود را افزایش دهد، که این امر باعث بهبود کیفیت و دقت پاسخ‌های زبانی مدل می‌شود.

در نهایت، مدل Paligemma با ترکیب معماری پیشرفته، تکنیک‌های بهینه‌سازی و قابلیت‌های مولد قوی، توانسته است جایگاه خود را به عنوان یکی از مدل‌های برجسته در حوزه بینایی-زبانی تثبیت کند. این مدل با ارائه پاسخ‌های دقیق و مرتبط به سوالات بصری، امکان استفاده گسترده‌ای در زمینه‌هایی مانند دستیارهای هوشمند، سیستم‌های مدیریت محتوا و تحلیل تصاویر پزشکی فراهم آورده است. همچنین، قابلیت مقیاس‌پذیری Paligemma آن را برای توسعه‌های آینده در زمینه هوش مصنوعی و یادگیری چندرسانه‌ای بسیار مناسب می‌سازد.

سوال اول:

زیربخش‌های سنتی تصویری یا متنی را توضیح دهید. همچنین، مدل‌های بینایی-زبانی (VLM) و تفاوت آن‌ها با سایر مدل‌ها را شرح دهید.

پاسخ:

۱. مدل‌های سنتی تصویری

مدل‌های سنتی تصویری عمدتاً بر پردازش و تحلیل داده‌های بصری مانند تصاویر و ویدئوها متمرکز هستند. این مدل‌ها از معماری‌های مختلفی بهره می‌برند که هر کدام برای وظایف خاصی طراحی شده‌اند.

الف) شبکه‌های عصبی پیچشی (CNNs)

- **معماری:** شبکه‌های عصبی پیچشی شامل لایه‌های پیچشی (Convolutional Layers)، لایه‌های فعال‌سازی (Activation Layers)، و لایه‌های جمع‌کننده (Pooling Layers) هستند.
- **عملکرد:** این شبکه‌ها ویژگی‌های محلی تصاویر را استخراج کرده و به تدریج به ویژگی‌های سطح بالاتر تبدیل می‌کنند.
- **کاربردها:** تشخیص اشیاء، طبقه‌بندی تصاویر، شناسایی چهره، و تحلیل ویدئو.

ب) ترانسفورمرهای بینایی (ViTs - Vision Transformers)

- **معماری:** برخلاف CNNها که از فیلترهای محلی استفاده می‌کنند، ViTs از مکانیزم توجه (Attention) بهره می‌برند تا وابستگی‌های بلندمدت در تصاویر را مدل‌سازی کنند.
- **عملکرد:** تصاویر را به تکه‌های کوچک (patches) تقسیم کرده و هر تکه را به یک بردار تعبیه می‌کند. سپس با استفاده از ترانسفورمرها، این بردارها را پردازش می‌کند.
- **کاربردها:** طبقه‌بندی تصاویر، تشخیص اشیاء، و سایر وظایف بینایی کامپیوتری.

۲. مدل‌های سنتی متنی

مدل‌های سنتی متنی بر پردازش داده‌های زبانی مانند متن‌های نوشتاری متمرکز هستند. این مدل‌ها از معماری‌های مختلفی بهره می‌برند که هر کدام برای وظایف خاصی طراحی شده‌اند.

الف) مدل‌های مبتنی بر RNN (شبکه‌های عصبی بازگشتی)

- **معماری:** شامل لایه‌های بازگشتی (Recurrent Layers) که به صورت توالی‌ای داده‌ها را پردازش می‌کنند.
- **عملکرد:** توانایی مدل‌سازی وابستگی‌های زمانی در داده‌های متنی.
- **کاربردها:** ترجمه ماشینی، پیش‌بینی کلمه بعدی، و تحلیل احساسات.

ب) مدل‌های مبتنی بر ترنسفورمر (Transformer-Based Models)

- **معماری:** استفاده از مکانیزم توجه (Attention Mechanism) برای پردازش توالی‌ها به صورت همزمان و بدون نیاز به پردازش ترتیبی.
- **عملکرد:** توانایی درک روابط پیچیده بین کلمات در یک جمله.
- **کاربردها:** ترجمه ماشینی، تولید متن، پاسخ به سوالات، و تحلیل معنایی.

۳. مدل‌های بینایی-زبانی (VLM) و تفاوت آن‌ها با مدل‌های سنتی

مدل‌های بینایی-زبانی (Vision-Language Models یا VLMs) ترکیبی از پردازش داده‌های بصری و متنی هستند که به مدل‌ها امکان می‌دهند تا به صورت همزمان به تحلیل و تولید محتوا در هر دو حوزه بپردازند. این مدل‌ها با استفاده از معماری‌های ترکیبی، قابلیت درک و تعامل میان تصاویر و متون را دارا هستند.

تفاوت‌ها با مدل‌های سنتی:

- **یکپارچگی داده‌ها:** برخلاف مدل‌های سنتی که تنها بر یک نوع داده (تصویری یا متنی) متمرکز هستند، VLMها به پردازش و تعامل میان داده‌های بصری و متنی می‌پردازند.
- **فضای تعبیه مشترک:** VLMها معمولاً داده‌های بصری و متنی را در فضای تعبیه مشترک نگاشت می‌کنند که امکان تعامل و ترکیب اطلاعات از هر دو حوزه را فراهم می‌آورد.
- **کاربردهای چندرسانه‌ای:** این مدل‌ها قادر به انجام وظایفی مانند پاسخ به سوالات بصری (VQA)، توصیف تصاویر، و تولید محتوای چندرسانه‌ای هستند که نیازمند درک ترکیبی از تصویر و متن می‌باشند.

مدل‌های معروف VLM:

1. **DALL-E:** مدل مولد که توانایی تولید تصاویر جدید بر اساس توضیحات متنی را دارد.
2. **Imagen:** مدل مولد با تاکید بر کیفیت بالاتر تصاویر تولید شده.
3. **Paligemma:** مدل چندمنظوره و مقیاس‌پذیر با استفاده از تکنیک‌های پیشرفته مانند PEFT و Flow Matching.

سوال دوم:

رویکردهای مختلف در مدل‌های بینایی-زبانی (VLM) را توضیح دهید. یکی از این رویکردها، رویکرد end-to-end است و رویکرد دیگر رویکرد ماژولار می‌باشد. مدل‌های مانند DALL·E و Paligemma را با این دو رویکرد توضیح دهید و مزایا و معایب هرکدام را به طور کلی مقایسه کنید.

پاسخ:

۱. معرفی رویکردهای End-to-End و ماژولار در مدل‌های بینایی-زبانی

در حوزه مدل‌های بینایی-زبانی (Vision-Language Models یا VLMs)، دو رویکرد اصلی برای طراحی و پیاده‌سازی مدل‌ها وجود دارد: رویکرد End-to-End و رویکرد ماژولار. هر یک از این رویکردها دارای ویژگی‌ها، مزایا و معایب خاص خود هستند که در ادامه به تفصیل به آن‌ها پرداخته می‌شود.

الف) رویکرد End-to-End

تعریف:

در رویکرد End-to-End، تمامی اجزای مدل از پردازش داده‌های ورودی (متنی و تصویری) تا تولید خروجی (مثلاً تصویر یا متن) به صورت یکپارچه و در یک معماری واحد آموزش داده می‌شوند.

ویژگی‌ها:

- **یکپارچگی کامل:** تمامی بخش‌های مدل به صورت همزمان و یکپارچه آموزش داده می‌شوند، بدون نیاز به تعاملات جداگانه بین ماژول‌ها.
- **سادگی پیاده‌سازی:** طراحی و پیاده‌سازی مدل ساده‌تر است زیرا نیازی به مدیریت تعاملات میان بخش‌های مختلف مدل نیست.
- **یادگیری همزمان:** مدل قادر است به طور همزمان از داده‌های متنی و تصویری برای بهبود عملکرد استفاده کند.

مزایا:

- **عملکرد بالا در وظایف ترکیبی:** توانایی یادگیری روابط پیچیده بین داده‌های متنی و تصویری به دلیل آموزش مشترک تمامی بخش‌ها.

- **خلاقیت و نوآوری:** قابلیت تولید خروجی‌های خلاقانه و متنوع بر اساس ترکیب داده‌های متنی و تصویری.
- **پایداری در آموزش:** مدل به دلیل آموزش همزمان تمامی بخش‌ها، ممکن است در یادگیری تعمیم‌پذیری بهتری داشته باشد.

معایب:

- **نیازمندی‌های محاسباتی بالا:** آموزش مدل‌های End-to-End معمولاً نیازمند منابع محاسباتی بسیار زیادی است.
- **انعطاف‌پذیری کمتر:** تغییر یا بهبود یک بخش خاص از مدل نیازمند بازآموزی کل مدل است.
- **چالش‌های تعمیم‌پذیری:** ممکن است در مواجهه با داده‌های جدید یا وظایف متفاوت عملکرد مناسبی نداشته باشد.

ب) رویکرد ماژولار

تعریف:

در رویکرد ماژولار، مدل به چندین بخش مستقل تقسیم می‌شود که هر کدام وظیفه خاصی را انجام می‌دهند و با یکدیگر از طریق رابط‌های مشخص تعامل می‌کنند.

ویژگی‌ها:

- **تفکیک وظایف:** هر ماژول به صورت مستقل برای انجام وظایف خاصی طراحی و آموزش داده می‌شود.
- **انعطاف‌پذیری بالا:** امکان تغییر یا بهبود هر بخش به صورت مستقل بدون نیاز به بازآموزی کل مدل.
- **مدیریت بهتر منابع:** می‌توان منابع محاسباتی را به طور بهینه‌تری تخصیص داد، زیرا هر ماژول می‌تواند به صورت جداگانه بهینه‌سازی شود.

مزایا:

- **انعطاف‌پذیری بالا:** قابلیت تغییر، بهبود یا جایگزینی هر بخش به صورت مستقل، بدون تأثیر بر سایر بخش‌ها.
- **مدیریت بهینه منابع:** امکان بهینه‌سازی و تخصیص منابع به هر ماژول بر اساس نیازهای خاص آن.
- **قابلیت توسعه:** افزودن ماژول‌های جدید برای وظایف جدید بدون تأثیر مستقیم بر سایر بخش‌ها امکان‌پذیر است.

معایب:

- پیچیدگی پیاده‌سازی: نیاز به طراحی رابط‌های مشخص و مدیریت تعامل میان ماژول‌ها که می‌تواند پیچیده باشد.
- هماهنگی بین ماژول‌ها: تضمین هماهنگی و تعامل بهینه میان بخش‌های مختلف مدل ممکن است چالش‌برانگیز باشد.
- کارایی کمتر در برخی موارد: عدم یکپارچگی کامل ممکن است در برخی وظایف خاص، کارایی مدل را کاهش دهد.

۲. مقایسه مدل‌های DALL·E و Paligemma با رویکردهای End-to-End و ماژولار

الف) مدل DALL·E (رویکرد End-to-End)

تعریف:

DALL·E یک مدل مولد بینایی-زبانی است که توسط OpenAI توسعه یافته و توانایی تولید تصاویر جدید بر اساس توضیحات متنی را دارد.

رویکرد:

DALL·E از رویکرد End-to-End استفاده می‌کند که در آن تمامی بخش‌های مدل از ورودی متنی تا خروجی تصویری به صورت یکپارچه آموزش داده می‌شوند.

ویژگی‌ها:

- معماری ترنسفورمر: استفاده از معماری ترنسفورمر برای ترکیب داده‌های متنی و تصویری.
- تولید مولد: توانایی تولید تصاویر خلاقانه و منحصر به فرد بر اساس توضیحات متنی.
- پیش‌تربیت گسترده: آموزش بر روی مجموعه داده‌های بزرگ تصاویر و توضیحات متنی.

مزایا:

- خلاقیت بالا: قابلیت تولید تصاویر نوآورانه و متنوع بر اساس توضیحات متنی.
- یکپارچگی متن و تصویر: توانایی درک و ترکیب دقیق بین داده‌های متنی و تصویری.
- پایداری در آموزش: مدل به دلیل آموزش همزمان تمامی بخش‌ها، قابلیت یادگیری روابط پیچیده بین متن و تصویر را دارا است.

معایب:

- نیازمندی‌های محاسباتی زیاد: آموزش و استنتاج مدل نیازمند منابع محاسباتی بالا است.
- انعطاف‌پذیری کم: تغییر یا بهبود یک بخش خاص از مدل نیازمند بازآموزی کامل مدل است.
- پتانسیل تولید ناصحیح: گاهی اوقات تصاویر تولید شده ممکن است دقیقاً با توضیحات متنی مطابقت نداشته باشند.

ب) مدل Paligemma (رویکرد ماژولار)

تعریف:

Paligemma یک مدل بینایی-زبانی چندمنظوره و مقیاس‌پذیر است که با استفاده از تکنیک‌های پیشرفته مانند PEFT (تنظیم دقیق با بهره‌وری پارامتری) و Flow Matching طراحی شده است.

رویکرد:

Paligemma از رویکرد ماژولار استفاده می‌کند که در آن مدل به بخش‌های مستقل مانند رمزگذار تصویری SigLIP و رمزگشا زبانی تقسیم می‌شود که هر کدام به صورت جداگانه بهینه‌سازی می‌شوند.

ویژگی‌ها:

- معماری مقیاس‌پذیر: قابلیت افزایش اندازه و پارامترهای مدل بدون کاهش کارایی، که امکان تعمیم‌پذیری و بهبود عملکرد را فراهم می‌آورد.
- تنظیم دقیق با PEFT: استفاده از تکنیک‌هایی مانند LoRA و QLoRA برای تنظیم دقیق مدل با مصرف حافظه و محاسبات کمتر.
- تطابق جریان (Flow Matching): استفاده از تکنیک‌های مولد پیشرفته برای بهبود کیفیت و کارایی تولید داده‌های چندرسانه‌ای.

مزایا:

- مقیاس‌پذیری بالا: امکان توسعه و تنظیم مدل برای وظایف خاص با مصرف منابع کمتر.
- کارایی بهینه: استفاده از تکنیک‌های PEFT باعث کاهش نیازمندی‌های محاسباتی و حافظه می‌شود، که امکان استفاده از مدل‌های بزرگ را در محیط‌های محدود فراهم می‌آورد.
- توانایی مولد قوی: تطابق جریان (Flow Matching) بهبود کیفیت و کارایی تولید داده‌های مولد را تضمین می‌کند.

- **انعطاف‌پذیری بالا:** امکان تغییر یا بهبود هر بخش به صورت مستقل بدون نیاز به بازآموزی کل مدل.

معایب:

- **پیچیدگی پیاده‌سازی:** نیاز به طراحی و مدیریت رابط‌های مشخص میان ماژول‌ها که می‌تواند پیچیده باشد.
- **هماهنگی بین ماژول‌ها:** تضمین هماهنگی و تعامل موثر میان بخش‌های مختلف مدل ممکن است چالش‌برانگیز باشد.
- **کمبود یکپارچگی:** ممکن است در برخی وظایف خاص، کارایی کمتر از مدل‌های End-to-End داشته باشد.

۳. مقایسه کلی بین مدل‌های Imagen و DALL-E

Imagen	DALL-E	ویژگی‌ها
End-to-End	End-to-End	رویکرد
ترنسفورمر پیشرفته‌تر	ترنسفورمر	معماری
بسیار بالا	خوب	کیفیت تصاویر
محدود	محدود	(Fine-Tuning) تنظیم دقیق
بالا	متوسط	پایداری
بسیار بالا	بالا	نیازمندی‌های محاسباتی
ندارد	ندارد	(Flow Matching) تطبیق جریان
کیفیت بالا، پایداری بیشتر	خلاقیت بالا، یکپارچگی متن و تصویر	مزایا
پیچیدگی بیشتر، نیاز به داده‌های بیشتر	نیاز به منابع زیاد، تولید ناصحیح	معایب

۴. نتیجه‌گیری

مدل‌های بینایی-زبانی مانند DALL-E و Imagen با استفاده از رویکرد End-to-End توانسته‌اند قابلیت‌های مولد قابل توجهی را ارائه دهند که شامل تولید تصاویر خلاقانه و با کیفیت بالا بر اساس توضیحات متنی است. این مدل‌ها با یکپارچگی کامل بین بخش‌های متنی و تصویری، توانایی یادگیری روابط پیچیده بین این دو نوع داده را دارا هستند. با این حال، نیازمندی‌های محاسباتی بالا و انعطاف‌پذیری محدود، از معایب اصلی این رویکردها به شمار می‌روند.

از سوی دیگر، مدل Paligemma با استفاده از رویکرد ماژولار و بهره‌گیری از تکنیک‌های پیشرفته مانند PEFT و Flow Matching، توانسته است تعادلی بین کارایی، مقیاس‌پذیری و کیفیت مولد برقرار کند. این مدل با امکان تنظیم دقیق و بهینه‌سازی بخش‌های مختلف به صورت مستقل، انعطاف‌پذیری بالاتری را ارائه می‌دهد و می‌تواند با منابع محاسباتی کمتر نیز کارایی مناسبی داشته باشد. با این حال، پیچیدگی پیاده‌سازی و نیاز به هماهنگی دقیق میان ماژول‌ها از چالش‌های اصلی این رویکرد به شمار می‌روند.

نتیجه نهایی:

انتخاب رویکرد مناسب بین End-to-End و ماژولار بستگی به نیازها، منابع موجود و ویژگی‌های خاص پروژه دارد. اگر تمرکز بر تولید تصاویر خلاقانه و با کیفیت بالا با منابع محاسباتی کافی باشد، مدل‌هایی مانند DALL-E و Imagen با رویکرد End-to-End می‌توانند گزینه‌های مناسبی باشند. اما اگر نیاز به انعطاف‌پذیری بیشتر، تنظیم دقیق و بهینه‌سازی منابع باشد، مدل‌هایی مانند Paligemma با رویکرد ماژولار می‌توانند انتخاب بهتری باشند.

زیر بخش دوم SIGLIP IMAGE ENCODER

در ادامه، متن پاسخ را با جزئیات کامل، به صورت روان و قابل فهم به زبان فارسی ارائه می‌کنیم:

سؤال اول

«معماری مدل چگونه است؟ فرایند آموزش فضای مشترک تعبیه شده (Joint Embedding Space) چگونه کار می‌کند؟ و از چه تابع ضرری (Loss Function) برای آموزش مدل SigLIP استفاده می‌شود؟»

پاسخ:

مدل **SigLIP (Sign Language Image Processing)** که در معماری **Paligemma** به کار می‌رود، در اصل نسخه‌ای تغییر یافته از مدل شناخته شده **CLIP (Contrastive Language-Image Pretraining)** است. هر دو مدل، تصاویر و داده‌های متنی را در یک فضای مشترک مرتب می‌کنند تا بتوان جفت‌های تصویر-متن مشابه را به راحتی مقایسه یا بازیابی کرد. تمرکز اصلی SigLIP روی تشخیص و پردازش زبان اشاره است، اما مبنای روش آن همان CLIP است.

در ادامه، مروری عمیق بر معماری، سازوکار آموزش، و تابع ضرر در SigLIP خواهیم داشت:

(Architecture) معماری (۱)

1. انکودر تصویر (Image Encoder):

○ در SigLIP، انکودر تصویر معمولاً از یک شبکه‌ی عصبی کانولوشنی (CNN) یا یک ویژن

ترنسفورمر (ViT) استفاده می‌کند:

■ CNN‌ها معماری‌های مرسوم برای پردازش تصویر هستند و با لایه‌های کانولوشن، ویژگی‌های سطح بالاتر را به تدریج از پیکسل‌های خام استخراج می‌کنند.

■ ViT، که رویکرد جدیدتری است، تصویر را همانند مجموعه‌ای از وصله‌ها (Patch) در نظر می‌گیرد (مشابه توکن‌ها در پردازش زبان طبیعی) و از مکانیزم ترنسفورمر برای بررسی ارتباط بین این قطعات تصویری استفاده می‌کند.

○ این انکودر، تصاویر ورودی را به بردارهای پرمحتوا و با ابعاد بالا تبدیل می‌کند که بیانگر محتوای معنایی تصویر هستند.

2. انکودر متن (Text Encoder):

○ انکودر متن معمولاً بر پایه‌ی یک مدل ترنسفورمری (نظیر BERT یا GPT) است. این مدل، توصیف‌های زبانی (یا برچسب‌های متنی) را پردازش کرده و آن‌ها را به یک فضای برداری با ابعاد بالا نگاشت می‌کند؛ فضایی که از نظر ابعاد با انکودر تصویر هم‌خوانی دارد.

○ این فرایند باعث می‌شود توصیف‌های متنی (مثلاً توضیحات زبان اشاره یا برچسب‌ها) به بردارهایی تبدیل شوند که بتوان آن‌ها را با بردارهای تصویری مقایسه کرد.

3. فضای مشترک تعبیه‌شده (Joint Embedding Space):

- نقطه‌ی کلیدی در CLIP و SigLIP، همین فضای مشترک است؛ جایی که هم انکودر تصویر و هم انکودر متن، ورودی‌های خود (تصویر و متن) را به فضای یکسانی نگاشت می‌کنند.
 - در این فضا، تصاویری که با یک توصیف متنی خاص مرتبط هستند، باید به هم نزدیک باشند و جفت‌های غیرمرتبط از هم فاصله بگیرند.
 - این ویژگی، امکان انجام کارهایی نظیر بازیابی میان‌مدلی (Cross-Modal Retrieval) را فراهم می‌کند؛ مثلاً یافتن تصاویر بر اساس متن یا یافتن متون بر اساس تصویر.
-

۲) آموزش فضای مشترک تعبیه‌شده (Training the Joint Embedding Space)

- داده‌های جفتی (Data Pairs):
مدل روی دیتاست‌های بزرگی آموزش می‌بیند که شامل جفت‌های تصویر-متن هستند. در SigLIP، این جفت‌ها مربوط به تصاویر زبان اشاره و توضیحات متنی (یا تفاسیر) آن‌هاست.
 - هدف آموزش (Training Objective):
در طول آموزش، هر دو انکودر تصویر و انکودر متن هم‌زمان بهینه می‌شوند تا تضمین کنند تعبیه‌های (بردارهای) مربوط به یک جفت تصویر-متن واقعی، هم‌تراز و به هم نزدیک باشند. به بیان دیگر، جفت‌های تصویر-متن مشابه باید در فضای مشترک، بردارهای نزدیک به هم داشته باشند و جفت‌های نامشابه از هم دور شوند.
-

۳) تابع ضرر (Loss Function)

- در SigLIP از تابع ضرر مقایسه‌ای (Contrastive Loss) استفاده می‌شود که ماهیتاً مشابه تابع ضرر مدل CLIP است.

- نحوه‌ی کارکرد تابع ضرر مقایسه‌ای:

در مدل‌های CLIP و SigLIP معمولاً از **Cross-Entropy متقارن** به شکل مقایسه‌ای استفاده می‌شود که دو خواسته‌ی اصلی را محقق می‌کند:

- **جفت‌های مثبت (Positive Pairs):** تعبیه‌های تصویر و متنی که به یکدیگر مرتبط هستند، باید در فضای تعبیه‌شده به هم نزدیک شوند. (کاهش فاصله‌ی کسینوسی یا افزایش شباهت کسینوسی)
- **جفت‌های منفی (Negative Pairs):** تعبیه‌های تصویر و متنی که به یکدیگر مرتبط نیستند، باید در این فضا از هم دور شوند. (افزایش فاصله‌ی کسینوسی یا کاهش شباهت کسینوسی)

- روش محاسبه (مثال ریاضی):

عموماً از توابعی مانند **Triplet Loss** یا **NT-Xent (Normalized Temperature-scaled Cross-Entropy)** استفاده می‌شود که شباهت هر جفت تصویر-متن را با تمام جفت‌های دیگر در یک بچ (Batch) مقایسه می‌کند. شکل کلی آن به صورت زیر است:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(I, T)/\tau)}{\sum_j \exp(\text{sim}(I, T_j)/\tau) + \sum_k \exp(\text{sim}(I_k, T)/\tau)}$$

که در آن:

- $\text{sim}(I, T)$ بیانگر شباهت کسینوسی بین بردار تصویر I و بردار متن T است.
- T پارامتری برای تنظیم مقیاس (دما) است؛ هرچه T کوچک‌تر باشد، تابع ضرر تمایل دارد تفاوت شباهت‌ها را واضح‌تر (شارپ‌تر) کند.
- در مخرج کسر، مجموع شباهت‌های همه‌ی تصویر-متن‌های دیگر در یک بچ قرار گرفته است. این کار باعث می‌شود مدل، فاصله‌ی بین جفت‌های مثبت را کم و فاصله‌ی بین جفت‌های منفی را زیاد کند.

به کمک یادگیری مقایسه‌ای، مدل می‌آموزد که شباهت بین جفت‌های واقعی (مثبت) را حداکثر و بین جفت‌های نامربوط (منفی) را حداقل کند؛ در نتیجه، یک فضای مشترک به‌خوبی هم‌راستا ساخته می‌شود.

سؤال دوم

«در فرایند آموزش SigLIP، ایده‌ای به کار گرفته شده است که باعث می‌شود آموزش سریع‌تری نسبت به CLIP داشته باشد. این ایده چیست و چه مزیتی دارد؟»

۱. جایگزینی تابع ضرر مبتنی بر Softmax با تابع ضرر سیگموئید جفتی (Pairwise Sigmoid)

مدل CLIP Contrastive Language–Image Pre Training از یک تابع ضرر مقایسه‌ای (Contrastive) مبتنی بر Softmax استفاده می‌کند که نیازمند ساختن و نگهداری یک ماتریس $(N \times N)$ از شباهت‌ها برای تمام تصاویر و متون در یک بچ (Batch) است. با بزرگ‌تر شدن اندازه‌ی بچ، این رویکرد از نظر حافظه و ارتباط بین دستگاه‌ها (All-Gather) بسیار پرهزینه می‌شود.

در مقابل، SigLIP Sigmoid Language–Image Pretraining از تابع ضرر سیگموئید جفتی بهره می‌گیرد و برای هر جفت تصویر-متن، به‌طور مستقل یک مسئله‌ی دودویی (مثبت یا منفی) حل می‌کند. این کار نیاز به نرمال‌سازی جهانی (Global Normalization) بر کل بچ را حذف کرده و ماتریس $(N \times N)$ را کنار می‌گذارد.

۲. تابع ضرر سیگموئید جفتی (Pairwise Sigmoid)

در این روش، مدل تلاش می‌کند برای هر جفت تصویر (I) و متن (T)، تشخیص دهد آیا این جفت «مطابقت صحیح» ($label = 1$) دارد یا «نامرتب» ($label = 0$). بنابراین:

{مثلاً بر اساس شباهت کسینوسی یا ضرب نقطه‌ای}

1. تابع سیگموئید:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

که خروجی آن در بازه‌ی (0,1) قرار دارد و نشان‌دهنده‌ی احتمال «مثبت بودن» جفت است.

2. تابع ضرر (Binary Cross-Entropy):

$$\text{loss} = \text{BCE}(\sigma(\text{sim}(I, T)), \text{label})$$

به طور مشروح، تابع ضرر سیگموئید جفتی به صورت زیر نوشته می‌شود:

$$\text{loss} = - \left[y \cdot \log(\sigma(\text{sim}(I, T))) + (1 - y) \cdot \log(1 - \sigma(\text{sim}(I, T))) \right]$$

که در آن:

○ y همان label یا برچسب جفت (0 یا 1) است.

○ $\sigma(\cdot)$ تابع سیگموئید است.

۳. مزایای استفاده از تابع ضرر سیگموئید در SigLIP

1. حذف نرمال‌سازی جهانی (Softmax) بر کل بچ

در CLIP، برای هر تصویر باید احتمال تعلق به هر متن موجود در بچ محاسبه شود (و بالعکس)، که نیازمند ماتریس (N×N) شباهت‌هاست.

○ در SigLIP، هر جفت مستقل بررسی می‌شود و نیازی به ماتریس بزرگ برای نرمال‌سازی نیست.

2. کاهش شدید مصرف حافظه

با کنار گذاشتن ماتریس $(N \times N)$ ، حافظه‌ی مورد نیاز برای پردازش شباهت‌های تصویر-متن بسیار کمتر می‌شود. این موضوع خصوصاً در اندازه‌های بچ بزرگ (۶۴k، ۳۲k، ۱۶k، و بیشتر) اهمیت حیاتی دارد.

3. کاهش عملیات مخابراتی (Communication Overhead)

- در رویکرد Softmax، همه‌ی تعبیه‌های تصویر و متن باید بین دستگاه‌های محاسباتی مبادله (All-Gather) شوند تا آن ماتریس کامل به دست آید.
- در روش سیگموئید جفتی، می‌توان به جای همه-برای-همه (All-Gather)، فقط به صورت بخشی (Chunked) یا تنها با دستگاه‌های همسایه تعبیه‌ها را ردوبدل کرد و جفت‌های منفی کافی را برای محاسبه‌ی ضرر به دست آورد.

4. پیاده‌سازی آسان‌تر و پایدارتر

- محاسبه‌ی $\log \sum \exp$ در Softmax نیازمند تثبیت (Stabilization) عددی است (مثلاً کم کردن ماکزیمم لاجیت‌ها).
- در سیگموئید، هر جفت تنها یک مقدار لجستیک تولید می‌کند که به شکل مستقیم در فرمول Binary Cross-Entropy قرار می‌گیرد و پیاده‌سازی آن ساده‌تر و پایدارتر است.

۴. نتایج عملی و تاثیر بر سرعت آموزش

- افزایش مقیاس‌پذیری:
با حذف نیاز به ماتریس $(N \times N)$ ، حال می‌توان اندازه‌ی بچ را بسیار بزرگ کرد (حتی تا صدها هزار یا یک میلیون) بی‌آنکه حافظه‌ی دستگاه‌ها زود تمام شود.
- بهبود سرعت و Throughput:
از آنجا که هم هزینه‌ی ارتباطی کاهش می‌یابد و هم نیازی به چندین پاس عددی نیست، SigLIP می‌تواند در مدت زمان کمتر به همان دقت یا حتی دقت بالاتر برسد.

- مدل‌های بسیار بزرگ روی سخت‌افزار محدود:

در مقاله اشاره شده که می‌توان روی تعداد معدودی TPU/GPU، اندازه‌ی بچ ۳۲k را بدون دردسر اجرا کرد. در CLIP، چنین اندازه‌ای معمولاً به خوشه‌ی بزرگ‌تری نیاز دارد.

۵. نتیجه‌گیری نهایی

ایده‌ی کلیدی در SigLIP این است که به‌جای اتکا به تابعی که برای هر تصویر باید در برابر تمام متون (و بالعکس) نرمال‌سازی شود (همان Softmax در CLIP)، هر جفت تصویر-متن را مستقل بررسی می‌کند و از تابع سیگموید جفتی برای تشخیص مثبت یا منفی بودن جفت استفاده می‌کند. این کار:

- مصرف حافظه را به‌شدت کاهش می‌دهد.
- ارتباط بین دستگاه‌ها را کم می‌کند.
- فرآیند را برای اندازه‌های بچ بسیار بزرگ مقیاس‌پذیر می‌کند.
- پیاده‌سازی و محاسبات را آسان‌تر و سریع‌تر می‌سازد.

فرمول نهایی تابع ضرر سیگموید جفتی در قالب Binary Cross-Entropy:

$$\text{loss}_{\text{sigmoid}}(I, T) = - \left[y \log(\sigma(\text{sim}(I, T))) + (1 - y) \log(1 - \sigma(\text{sim}(I, T))) \right].$$

در این‌جا:

- I و T نماینده‌ی تعبیه‌های تصویر و متن هستند.
- $\text{sim}(I, T)$ یک تابع شباهت (مثلاً ضرب نقطه‌ای یا کسینوسی).
- σ همان تابع سیگموید است.
- $y \in \{0, 1\}$ برچسب مثبت (۱) یا منفی (۰) بودن جفت است.

پرسش ۱

پس از آنکه انکودر تصویر و دیکودر متن در داده‌های تک‌وجهی (uni-modal) جداگانه آموزش دیدند، در مرحله بعد این دو مدل با هم ترکیب یا در معماری کلان مدل قرار می‌گیرند و در دو مرحله دیگر دوباره آموزش (pre-train) می‌شوند. این دو مرحله آموزش چگونه انجام می‌شود و هر کدام چه ضرورتی دارند؟

مروری کلی بر مراحل (Overview)

1. Stage0 (آموزش تک‌وجهی - Unimodal Pre Training)

- خلاصه: مدل بینایی (Vision Encoder) و مدل زبانی (Language Model) هرکدام جداگانه روی داده‌های وسیع مربوط به حوزه خودشان آموزش می‌بینند. مثلاً SigLIP فقط با جفت‌های تصویر-متن (برای یادگیری بازنمایی تصویری) به شکل کنتراستيو (contrastive) تمرین می‌شود و Gemma روی دیتاست‌های متنی بزرگ، مدل زبان خودبازگشتی را یاد می‌گیرد.
- هدف: اطمینان از آنکه هر کدام از این بخش‌ها (تصویری یا زبانی) به‌تنهایی قدرتمند شوند و در کار تک‌وجهی خود، عملکردی قوی ارائه دهند. در نتیجه، در مرحله بعدی هنگامی که این دو کنار هم قرار می‌گیرند، پایه اولیه‌شان خوب است و نیاز نیست همه‌چیز از صفر یاد گرفته شود.

2. Stage1 (پیش‌آموزش چندوجهی - Multimodal Pre Training)

- خلاصه: ترکیب SigLIP (به‌عنوان انکودر تصویر) و Gemma-2B (به‌عنوان دیکودر زبانی) در یک مدل واحد. از این به بعد، مدل کلان با تسک‌های چندوجهی (متنوع) آموزش می‌بیند. در این مرحله رزولوشن ورودی معمولاً 224×224 پیکسل است.
- جزئیات مهم:
 - بدون فریزکردن کامل بخش‌ها: برخلاف روالی که گاهی انکودر تصویر یا مدل زبان فریز می‌شود تا خدشه‌ای در پارامترهای پیش‌آموزش دیده وارد نشود، در PaliGemma کل

پارامترها (هم تصویری و هم زبانی) قابل به روزرسانی هستند. این کار باعث می شود مدل بتواند یاد بگیرد که تصویر و متن را به شکلی عمیق تر تلفیق کند.

■ آغاز ملایم (Warm-up) برای نرخ یادگیری انکودر تصویر:

■ اگر از همان ابتدا نرخ یادگیری بزرگی برای بخش تصویری بگذاریم، ممکن است «گرادیان های نامربوط» از سمت زبان، کیفیت انکودر تصویر را شدیداً خراب کنند. برای جلوگیری، ابتدا نرخ یادگیری انکودر تصویر خیلی کم تنظیم می شود و به تدریج زیادتر می گردد.

■ در نتیجه، سیگنال های ناهمخوانی در مراحل ابتدایی کمتر به انکودر آسیب می زنند و به تدریج مدل دو وجهی، همگرایی بهتری پیدا می کند.

■ استفاده از چندین تسک مختلف: در دیتاست های چندوجهی، علاوه بر کپشن نویسی

(captioning) و پرسش پاسخ تصویری (VQA)، تسک هایی مثل تشخیص اشیا، سگمنت کردن بخش های تصویر، تشخیص متن در عکس (OCR) و غیره نیز گنجانده می شوند. این تنوع به مدل کمک می کند مهارت های گسترده ای از «درک تصویر و زبان» را به دست آورد.

■ طراحی معماری:

■ ابتدا خروجی انکودر تصویر (توکن های تصویری) از یک لایه خطی (linear projection) عبور می کند تا از لحاظ ابعاد با توکن های زبانی (واژگان Gemma) یکسان شود.

■ سپس، توکن های تصویر + BOS + توکن های پیشوند (SEP) + prefix + توکن های پسوند EOS + suffix به مدل زبان (به صورت یک دنباله) فرستاده می شود.

■ **قانون توجه (attention)** به صورت prefix-LM طراحی شده است؛ یعنی توکن های تصویر و توکن های prefix می توانند همدیگر را ببینند (دوسو) ولی توکن های suffix فقط به گذشته خودشان دسترسی دارند (خودبازگشتی). این الگو برای مولد بودن مدل ضروری است (یعنی مدل بتواند عبارت نهایی را پیش بینی کند).

○ **هدف:** مدل حالا می تواند تصویر و متن را توأم پردازش کند و روی مجموعه ای عظیم از وظایف چندوجهی تمرین ببیند؛ بنابراین مهارت همبستگی بین «بردارهای بصری» و «تعبیر زبانی» شکل می گیرد. در پایان Stage1، مدل هم می تواند صفر تا صد تصویر را بخواند (توسط انکودر SigLIP) و هم می تواند پاسخی در قالب زبان (با Gemma) تولید کند.

3. Stage2 (افزایش رزولوشن - Resolution Increase)

- **خلاصه:** همان مدل آموزش دیده Stage1، حالا با رزولوشن تصویری بالاتر (معمولاً 448×448 یا 896×896) برای مدت محدودی ادامه داده می‌شود.

- **جزئیات مهم:**

- **تسک‌های رزولوشن حساس:** در این مرحله بیشتر داده‌های آموزشی مربوط به خواندن متن ریز (OCR)، نمودارها و دیاگرام‌ها، همچنین وظایف مربوط به اجسام کوچک یا تشخیص/سگمنت کردن اشیای ظریف در اولویت بالاتری قرار می‌گیرند. چون این وظایف واقعاً به پیکسل‌های بیشتری احتیاج دارند و در 224×224 ممکن است اطلاعات کافی در دسترس نباشد.

- **تمرین نسبتاً کوتاه:** بر خلاف Stage1 که ممکن است چند صد میلیون یا حتی میلیارد نمونه را ببیند، این مرحله فقط ده‌ها میلیون نمونه جدید می‌بیند (اما گران‌تر هستند چون رزولوشن بالاتری دارند). ایده این است که مدل «دانش چندوجهی» کلی خود را در Stage1 فراگرفته و حالا فقط «یاد می‌گیرد» چطور با جزئیات تصویری بیشتر برخورد کند.

- **فایده افزایش توکن تصویر:** وقتی تصویر بزرگ‌تر می‌شود، به تبع تعداد توکن‌های تصویری (خروجی انکودر ViT) نیز بیشتر می‌شود. این یعنی مدل ظرفیت بیشتری دارد تا اطلاعات موضعی دقیق‌تری را در کد توکن‌ها بگنجاند. نمود این قضیه در وظایف OCR، سگمنت‌سازی و غیره دیده می‌شود.

- **هدف:** بهتر شدن مدل در وظایف ریزدانه (fine-grained). در عمل، مقاله نشان می‌دهد که عملکرد مدل روی خواندن متن‌های کوچک، دیزاین‌های ال، مسائل چالش‌برانگیز در VQA (مثلاً اطلاعات دقیق مثل اعداد روی چارت یا جزئیات شیء) با عبور از Stage2 ارتقا می‌یابد.

4. Stage3 (انتقال نهایی - Transfer)

- **خلاصه:** نتیجه کل مراحل (انکودر + دیکودر) حالا به صورت یک مدل چندوجهی پایه در دسترس است. اما هنوز ممکن است نیاز باشد آن را برای یک تسک یا کاربرد خاص «fine-tune» کنیم.

- **چرایی نیاز به Fine-tuning:**

- گاهی می‌خواهیم تسک‌های ویژه‌ای را بهتر حل کنیم؛ مثلاً Video QA، یا مثلاً دیتاست یک رشته خاص مثل تصاویر پزشکی. یا اصلاً بخواهیم مدل را اینستراکشن-تیون (instruction tuning) کنیم تا بتواند به سبک گفت‌وگویی عمل کند.

■ بنا بر نتایج مقاله، همین مدل پایه را می‌توان در مدت کوتاهی روی دیتاست تخصصی‌ای (مثلاً COCO Caption، InfographicQA، WidgetCap و ...) تنظیم کرد و عملکردی بالا به دست آورد.

پرسش ۲

در معماری Pali Gemma، دنباله ورودی به مدل زبان (Gemma) به دو بخش اصلی «توکن‌های پیشوند (prefix)» و «توکن‌های پسوند (suffix)» تقسیم می‌شود. این تقسیم‌بندی ریشه در نحوه تعامل «داده ورودی» با «سیستم تولید متن خودبازگشتی» دارد. در ادامه، بدون آوردن مثال صریح، به صورت مفهومی و فنی نقش و ضرورت هر کدام را توضیح می‌دهیم:

۱. توکن‌های پیشوند (Prefix Tokens)

الف) شناسنامه تسک و شرایط ورودی

توکن‌های پیشوند عموماً برای توضیح کاری است که مدل باید انجام دهد. این کار ممکن است «پرسش پاسخ تصویری»، «کپشن‌نویسی»، «تشخیص اشیا»، یا هر وظیفه دیگری باشد. از آنجا که در PaliGemma یک مدل زبان بزرگ (LLM) مسئول تولید متن است، باید بداند چه کاری از او خواسته شده است. پس در بخش پیشوند، کلیدواژه‌ها یا عبارت‌های مربوط به نوع وظیفه، زبان هدف، یا دیگر توضیحات آورده می‌شود.

ب) توجه دوسویه به تصویر و پیشوند

از آنجا که معماری PaliGemma برای مرحله ورودی از رویکرد «prefix-LM» استفاده می‌کند، توکن‌های پیشوند از ابتدا می‌توانند با همدیگر (و با توکن‌های تصویری) دوسویه تعامل داشته باشند؛ یعنی به یکدیگر و به داده‌های بصری نگاه کنند و لایه‌های توجه (attention) هرکدام را ببینند. به این ترتیب مدل می‌تواند بهتر تصمیم بگیرد که چه جنبه‌هایی از تصویر یا چه سبک پاسخی را باید دنبال کند.

ج) تفکیک تسک‌های مختلف

زمانی که مدل روی دیتاست‌های چندوجهی متنوع آموزش می‌بیند، این توکن‌های پیشوند به نوعی «برچسب» یا «مسیریاب» هستند تا به مدل نشان دهند الان چه نوع وظیفه‌ای در حال انجام است. این موضوع جلوی تداخل آموزش در وظایف مختلف را می‌گیرد و از همان ابتدا، جهت‌گیری مدل را مشخص می‌کند.

د) نیاز به عدم پیش‌بینی خودبازگشتی

در روش prefix-LM، فرض بر این است که همه توکن‌های پیشوند و توکن‌های تصویری، داده‌ای هستند که «از قبل» داریم و نباید آن‌ها را به صورت خودبازگشتی پیش‌بینی کنیم. یعنی این توکن‌ها بخشی از «کانتکست» هستند و قرار است تمامشان را مدل ببیند تا بر اساسشان پاسخ نهایی را بسازد. بنابراین، توکن‌های پیشوند یک نقش زمینه‌ساز یا context-provider ایفا می‌کنند.

۲. توکن‌های پسوند (Suffix Tokens)

الف) تولید مرحله به مرحله (خودبازگشتی)

مهم‌ترین ویژگی توکن‌های پسوند آن است که مدل زبان باید آن‌ها را «مرحله به مرحله» بسازد. این تفاوت بنیادین با بخش پیشوند دارد که از قبل مشخص و ثابت بود. در بخش پسوند، مدل فقط می‌تواند به توکن‌هایی که تاکنون تولید کرده نگاه کند (همچنین به کل پیشوند و تصویر)، اما اجازه ندارد آینده پاسخ را بداند. این ساختار «خودبازگشتی» هسته اصلی مدل‌های زبانی مولد است.

ب) خروجی اصلی مدل

هر کاری که از مدل خواسته شود (شرح، پاسخ، لیستی از مختصات، متن ترجمه شده و ...) در این ناحیه شکل می‌گیرد. مدل از روی محتوای بصری و متن پیشوند، تصمیم می‌گیرد هر توکن بعدی چه باشد و آن را می‌نویسد. به این ترتیب، کل «پاسخ» در دنباله پسوند قرار می‌گیرد تا سرانجام به پایان برسد.

ج) عدم دید دوسویه

برخلاف پیشوند که توکن‌های آن می‌توانند به شکل دوسویه یکدیگر را ببینند، در بخش پسوند مدل اجازه ندارد به توکن‌های «آینده» نگاه کند؛ یعنی یک توکن پسوندی فقط از پیشوند + تصویر + توکن‌های پسوندی‌ای که قبلاً تولید شده‌اند، خبر دارد. این ممنوعیت «نگاه به آینده» بخشی از مکانیسم اصلی زبان مدل خودبازگشتی است و تضمین می‌کند تولید متن پله پله پیش می‌رود.

د) پایان (EOS) و پدکردن (PAD)

برای متوقف کردن فرایند تولید، نشانگر پایان (EOS) در انتهای پسوند قرار می‌گیرد. این یعنی مدل می‌گوید «کار تمام شد» و اگر لازم باشد، با توکن‌های PAD دنباله را تا طول ثابتی پر می‌کنند. بنابراین، پسوند بخشی است که مدل زبان در آن آزاد است هر چقدر نیاز دارد محتوای پاسخ را ایجاد کند تا به نتیجه برسد.

۳. چرایی تمایز پیشوند و پسوند در PaliGemma

الف) سازگاری با رویکردهای چندوجهی

مدل ابتدا توکن‌های تصویر و پیشوند را به‌طور آزاد می‌بیند، پس می‌تواند تمام اطلاعات محیطی، دستوری، و تصویری را یکجا درک کند. آنگاه در قالب پسوند، به‌صورت پله‌پله واکنش نشان می‌دهد و متن خروجی را تولید می‌کند. این ساختار هم با ماهیت «پردازش تصویر» سازگار است (که پیشاپیش داده شده) و هم با ماهیت «زبان خودبازگشتی» در بخش تولید خروجی.

ب) پشتیبانی از طیف متنوع کارکردها

زمانی که لازم باشد مدل زبان در تسک‌های مختلف (کپشن، پرسش‌پاسخ، تشخیص اشیا و ...) شرکت کند، به‌سادگی در بخش پیشوند دستور کار را تغییر می‌دهیم؛ اما بخش پسوند همچنان جایگاه تولید متن است که مطابق آن دستور کار پیش می‌رود.

ج) عدم تداخل دستورالعمل و پاسخ

اگر دستورالعمل (سؤال یا هر قالبی) و پاسخ در هم آمیخته می‌بودند، مدل باید تشخیص می‌داد چه قسمتی از ورودی «باید پیش‌بینی شود» و چه قسمتی «صرفاً ورودی ثابت» است. اما با جداکردن پیشوند و پسوند، این معضل حل می‌شود: پیشوند همیشه ورودی است و پیش‌بینی‌نشده، پسوند همیشه خروجی است و باید مرحله‌به‌مرحله پیش‌بینی شود.

۴. فواید عملی

1. همگرایی پایدار

از آنجا که ساختار ماسک‌گذاری در بخش پاسخ خود بازگشتی و در بخش پرسش/تصویر دو سوست، مدل به شکل نسبتاً ساده و قابل‌کنترلی آموزش می‌بیند. بخشی از خطاها و ناسازگاری‌های احتمالی که در معماری‌های پیچیده پیش می‌آمد، کم می‌شود.

2. استفاده از دانش پیشوند برای رمزگشایی بهتر

وقتی سؤال یا دستوری در پیشوند است، همه توکن‌های پسوند می‌توانند به آن دسترسی پیدا کنند. به محض تولید هرتوکن در پسوند، از محتوای پیشوند (شامل مفهوم وظیفه، زبان، دستورات خاص و ...) بهره می‌گیرد. این جریان اطلاعات باعث می‌شود پاسخ با نیاز تسک هماهنگ باشد.

3. توسعه‌پذیری و افزودن وظایف جدید

در آینده، اگر وظیفه یا ساختار جدیدی تعریف کنیم، کافی است کلیدواژه یا فرمت جدیدی در پیشوند اضافه کنیم تا مدل بداند کار چیست. خروجی باز هم در پسوند خواهد بود. به این شکل، معماری انعطاف‌پذیری بالایی پیدا می‌کند.

پرسش ۳

یکی از چالش‌های معمول در چنین ساختارهایی این است که مدل زبان (Language Model) روی توکن‌های متنی تنظیم شده و وقتی با داده‌های دیداری مواجه می‌شود ممکن است کیفیتش افت کند. مدل PaliGemma چگونه این مشکل را حل کرده است؟

در بسیاری از رویکردهای چندرسانه‌ای (Multimodal) وقتی یک مدل زبان بزرگ (LLM) را مستقیماً با ورودی بصری درهم می‌آمیزیم، احتمال دارد که «دانش زبانی» از قبل آموخته‌شده تحت تأثیر سیگنال‌های تصویری و گرادیان‌های جدید، تضعیف شود یا حتی بخشی از آن فراموش شود. اصطلاحاً این پدیده را "Catastrophic Forgetting" یا «فراموشی فاجعه‌بار» می‌نامند. در چنین شرایطی، مدل شاید هنوز بتواند اطلاعاتی از تصویر استخراج کند، اما هم‌زمان مهارت‌های زبانی از بین می‌روند و یا کیفیت کلی آن افت می‌کند.

در مدل PaliGemma (و روش‌های مشابه)، چندین راهکار هم‌زمان به کار گرفته می‌شود تا از تخریب یا نزول عملکرد LLM هنگام دریافت داده تصویری جلوگیری شود:

1. استفاده از انکودر تصویری پیش‌آموزش‌دیده قدرتمند

- **SigLIP**: این انکودر تصویر از قبل (در Stage0) با یک دیتاست عظیم از جفت‌های تصویر-متن و به روش کنتراستيو (Contrastive Learning) آموزش دیده است.
 - **مزیت اصلی**: وقتی مدل زبان به جای تصویر خام، بردارهای باکیفیت و معنادار را دریافت می‌کند، نیاز ندارد «از صفر» رمزگشایی پیکسل‌ها را یاد بگیرد. بنابراین، سیگنال بصری ورودی، ساختارمند و به ثبات رسیده است و کمتر باعث تغییر ناگهانی در پارامترهای زبان‌مدل می‌شود.
-

2. طراحی "Warm-up" ملایم برای نرخ یادگیری بخش تصویری

- در مرحله چندوجهی (Stage1)، اگر از همان ابتدا نرخ یادگیری (LR) برای انکودر تصویر خیلی بزرگ باشد، مدل زبان که هنوز هماهنگ با سیگنال تصویری نیست، ممکن است گرادیان‌های عجیب و غریب به سمت انکودر تصویر بفرستد.
 - **راه‌حل**: ابتدا LR انکودر تصویر را خیلی کم می‌گذارند و رفته‌رفته به مقدار دلخواه می‌رسانند.
 - **نتیجه**:
 1. در مراحل اولیه که مدل زبان هنوز «تطابق» کافی با داده‌های تصویری ندارد، تغییرات کمتری روی بخش تصویری اعمال می‌شود (جلوگیری از تخریب).
 2. به محض آنکه مدل زبان و سیگنال چندرسانه‌ای تا حدی منسجم شدند، LR انکودر تصویر بالاتر رفته و می‌تواند در راستای وظایف پیچیده (مانند رابطه‌های فضایی یا جزئیات ظریف) یادگیری را بهبود بخشد.
-

3. ساده نگه‌داشتن اتصال (Connector) بین انکودر تصویر و مدل زبان

- **لایه خطی (Linear Projection)**:
 - بسیاری از کارها، برای یکپارچه‌سازی ورودی تصویر به مدل زبان، یک شبکه MLP چندلایه یا سازوکارهای پیچیده‌تر به کار می‌گیرند.
 - در PaliGemma، از یک لایه خطی ساده برای تبدیل ابعاد خروجی انکودر تصویر به همان ابعاد توکن‌های زبانی استفاده می‌شود.

- **مزیت:** سادگی این لایه هم باعث پایداری بیشتر در آموزش می‌شود، هم ریسک «همه‌چیز را بهم ریختن» را کاهش می‌دهد. هرچه لایهٔ میانی حجیم‌تر باشد، احتمال دارد مدل زبان در آن «گیر بیفتد» و شکل ناهنجاری از همگرایی یا حتی اُفت کلی ایجاد شود.
-

4. ساختار ماسک‌گذاری از نوع Prefix-LM

- در معماری‌های «Encoder-Decoder» سنتی، ممکن است برخی توکن‌های ورودی نیز به شکل خودبازگشتی پیش‌بینی شوند یا بخش بصری و متنی توزیع ماسک پیچیده داشته باشند.
 - در **PaliGemma**، به روش Prefix-LM عمل می‌شود:
 1. توکن‌های تصویر + توکن‌های پرسش/دستور (prefix) را می‌توان به صورت دوسو (bidirectional) مشاهده کرد. یعنی این توکن‌ها همدیگر را می‌بینند و با هم «تعمق» می‌کنند.
 2. اما توکن‌های پاسخ (suffix) به صورت خودبازگشتی تولید می‌شود: هر توکن بعدی با توجه به توکن‌های قبلی‌اش نوشته می‌شود.
 - چرا کمک می‌کند؟
 1. توکن‌های تصویر از همان آغاز می‌توانند «بدانند» سؤال یا دستور متنی چیست (prefix). پس بردارهای تصویری به شکلی هدفمند، سیگنال لزوم توجه به قسمت‌های خاص تصویر را می‌گیرند.
 2. مدل زبان هم فقط در بخش پاسخ (suffix) هست که باید مرحله‌به‌مرحله خروجی تولید کند؛ بنابراین «مغز زبانی» دچار پردازش عجیب کل ورودی (شامل بخش تصویری) به صورت خودبازگشتی نمی‌شود. در نتیجه، دانش زبانی کمتر لطمه می‌خورد.
-

5. عدم فریز کامل بخش زبان

- برخلاف برخی روش‌ها که می‌گویند «برای حفاظت از مدل زبان، آن را فریز کنیم و فقط بر بخش تصویری تنظیمات انجام شود»، در PaliGemma این‌طور نیست.
- اما: چون SigLIP قبلاً قدرتمند است و ضمناً warm-up ملایم دارد، گرادیان‌ها به شکلی هدایت می‌شوند که مدل زبان تضعیف نشود.
- فایده:

1. LLM می‌تواند از سیگنال تصویری یاد بگیرد که مثلاً چگونه به تصاویر پیچیده پاسخ زبانی دقیق دهد (مثلاً کجاها باید شمارش کند، کجاها باید توصیف کند...).
2. مدل زبان توانایی‌های خود در درک معنایی و زبانی را حفظ می‌کند و حتی غنی‌تر هم می‌شود (چون حالا از تصاویر هم دانش بیشتری کسب می‌کند).

6. جلوگیری از «همه‌چیز را یک‌دفعه با هم تمرین دادن» (تدریج در طراحی مراحل)

- رویکرد PaliGemma کاملاً مرحله‌بندی شده است:
 - Stage0 (آموزش تک‌وجهی) → Stage1 (چندساعته یا چندروزه، ولی تنوع زیاد داده‌های تصویری-متنی) → Stage2 (رزولوشن بالا، ولی مدت کمتر).
 - این خودش جلوی فشار بیش‌ازحد را می‌گیرد که ممکن است مدل زبان با ورودی‌های خام/حجیم‌بصری از ابتدا له شود.
- در انتها، مدلی داریم که نه تنها افت نمی‌کند بلکه در انواع وظایف بینایی-زبانی خوب عمل می‌کند.

زیر بخش چهارم - Transfer Learning

در این بخش ابتدا به تعریف PEFT می‌پردازیم :

PEFT: Parameter-Efficient Fine-Tuning

PEFT مخفف Parameter-Efficient Fine-Tuning است که به یک رویکرد نوآورانه در تنظیم (Fine-Tuning) مدل‌های بزرگ یادگیری عمیق اشاره دارد. این روش به جای به‌روزرسانی تمام پارامترهای یک مدل بزرگ، تنها زیرمجموعه‌ای کوچک از پارامترها را تغییر می‌دهد. هدف اصلی PEFT کاهش منابع محاسباتی، حافظه‌ی مورد نیاز، و هزینه‌ی تنظیم مدل است، بدون کاهش چشمگیر در دقت یا عملکرد.

دلایل استفاده از PEFT

1. اندازه‌ی مدل‌های بزرگ:

- مدل‌های مدرن یادگیری عمیق (مانند BERT، GPT-3، یا ViT) معمولاً شامل میلیاردها پارامتر هستند.
- تنظیم این مدل‌ها به طور کامل (Full Fine-Tuning) مستلزم حافظه و محاسبات بسیار زیاد است.

2. هزینه و زمان بالا:

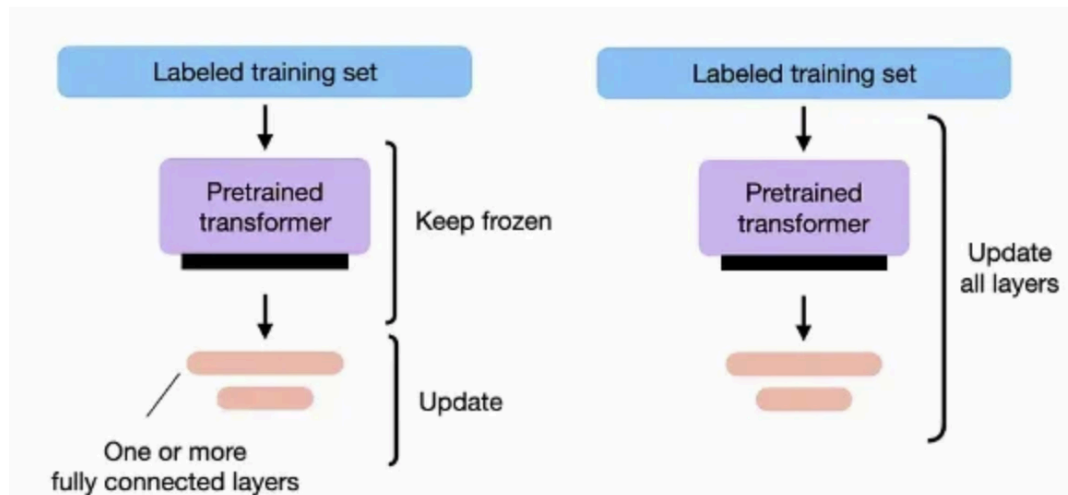
- تنظیم کامل مدل‌های بزرگ برای هر وظیفه نیازمند منابع سخت‌افزاری قوی و زمان زیادی است که ممکن است برای بسیاری از کاربران عملی نباشد.

3. افزایش قابل استفاده بودن مدل‌ها:

- با PEFT، می‌توان مدل‌های از پیش‌آموزش‌دیده را برای کاربردهای خاص با تغییرات جزئی تنظیم کرد، بدون نیاز به بازآموزی کل مدل.

اصول PEFT

Full finetuning vs PEFT



در PEFT، تنها بخش کوچکی از پارامترهای مدل تغییر می‌کنند یا پارامترهای اضافی به مدل اضافه می‌شوند، در حالی که بخش عمده‌ی پارامترها ثابت می‌ماند. این روش معمولاً شامل مراحل زیر است:

1. مدل اصلی را ثابت نگه دارید (Frozen Model):

- پارامترهای اصلی مدل از پیش آموزش دیده، ثابت نگه داشته می شوند و تنها بخشی خاص (معمولاً کوچک) از آن ها تغییر می کند.

2. اضافه کردن لایه های اضافی (اختیاری):

- در برخی از روش های PEFT، لایه ها یا ماژول های کوچک جدیدی (مانند **Adapters**) به مدل اضافه می شوند که وظیفه ی یادگیری تغییرات خاص وظیفه را بر عهده دارند.

3. به روزرسانی انتخابی:

- به جای تغییر کل پارامترها، تنها بخش خاصی از مدل (مثلاً لایه های آخر، یا ماژول های خاص) به روزرسانی می شوند.

روش های معروف PEFT

1. Adapters:

- در این روش، لایه های کوچکی به بین لایه های اصلی مدل اضافه می شوند. این لایه ها وظیفه دارند تغییرات مربوط به وظیفه ی خاص را یاد بگیرند.
- پارامترهای اصلی مدل ثابت باقی می مانند و تنها پارامترهای Adapter به روزرسانی می شوند.

2. LoRA (Low-Rank Adaptation):

- یک روش موثر در PEFT است که فرض می کند تغییرات مورد نیاز برای تنظیم مدل را می توان به یک فضای کم رتبه (Low-Rank) محدود کرد.
- LoRA تنها بخشی از پارامترهای مدل را تغییر می دهد، معمولاً با افزودن ماتریس های کم رتبه به مدل.

3. Prefix-Tuning:

- در این روش، بردارهای اضافی (Prefixes) به ورودی مدل اضافه می‌شوند که مدل را برای وظیفه‌ی خاص هدایت می‌کنند.
- این بردارها آموزش داده می‌شوند، اما خود مدل ثابت باقی می‌ماند.

4. Prompt-Tuning:

- شبیه به Prefix-Tuning، اما فقط روی تنظیمات ورودی (Prompts) تمرکز دارد. مدل اصلی هیچ تغییری نمی‌کند.

5. BitFit:

- این روش تنها بایاس‌ها (Biases) مدل را تنظیم می‌کند و تمام پارامترهای دیگر را ثابت نگه می‌دارد.

6. Diff Pruning:

- این روش فرض می‌کند که تفاوت (Difference) بین مدل اولیه و مدل تنظیم‌شده را می‌توان به صورت پراکنده (Sparse) ذخیره کرد.

مزایای PEFT

1. کاهش منابع محاسباتی:

- چون تنها بخش کوچکی از پارامترها به‌روزرسانی می‌شوند، نیاز به حافظه و محاسبات کاهش می‌یابد.

2. سرعت بالاتر در تنظیم:

- تنظیم مدل‌های بزرگ زمان‌بر است، اما PEFT این فرآیند را سریع‌تر می‌کند.

3. صرفه‌جویی در ذخیره‌سازی:

- به‌جای ذخیره‌ی کل مدل تنظیم‌شده، تنها تغییرات کوچک ذخیره می‌شوند.

4. افزایش انعطاف‌پذیری:

مدل اصلی می‌تواند برای چندین وظیفه‌ی مختلف، بدون تغییر کلی، استفاده شود.

5. دسترسی بیشتر به مدل‌های بزرگ:

- PEFT امکان استفاده از مدل‌های بزرگ را برای کاربران با منابع محدود فراهم می‌کند.

معایب PEFT

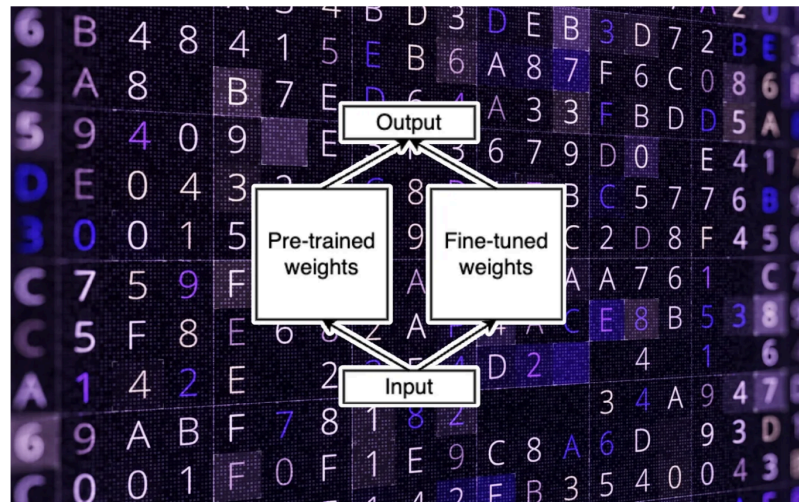
1. محدودیت در تنظیمات پیچیده:
 - ممکن است برای وظایف پیچیده یا داده‌های کم، عملکرد آن به اندازه‌ی تنظیم کامل نباشد.
2. نیاز به طراحی دقیق:
 - انتخاب قسمت‌هایی از مدل که باید به‌روزرسانی شوند، نیازمند دانش عمیق از مدل و وظیفه است.
3. کارایی کمتر در برخی موارد:
 - اگر تغییرات گسترده در مدل برای وظیفه‌ی خاص لازم باشد، PEFT ممکن است به اندازه‌ی Full Fine-Tuning مؤثر نباشد.

مثال کاربردی

فرض کنید شما یک مدل بزرگ مانند **GPT-3** دارید و می‌خواهید آن را برای تحلیل احساسات (Sentiment Analysis) تنظیم کنید. با استفاده از PEFT:

1. مدل اصلی ثابت می‌ماند و فقط لایه‌های اضافه‌شده یا بایاس‌ها به‌روزرسانی می‌شوند.
2. حافظه و محاسبات بسیار کمتری نیاز دارید، زیرا تنها تعداد کمی از پارامترها تغییر می‌کنند.
3. مدل نهایی سبک‌تر و سریع‌تر خواهد بود، و به راحتی می‌توانید آن را برای سایر وظایف نیز استفاده کنید.

LoRA



LoRA approach. Source:[6]

پرسش اول :

در این پاسخ، هر کدام از بخش‌های مطرح‌شده دربارهٔ QLoRA، LoRA، پارامتر **رنگ (Rank)** در LoRA، و دیتاتایپ **NF4** را با جزئیات بیشتر تشریح می‌کنیم تا روشن شود در عمل چه اتفاقی می‌افتد و هر کدام چه اثری دارند.

۱. LoRA Low-Rank Adaptation

۱.۱. ایده اصلی در LoRA

- وقتی می‌خواهیم یک مدل بزرگ زبانی (LLM) یا مدل ویژن-زبان را برای یک تسک تازه **فاین‌تیون** کنیم، معمولاً نیاز به به‌روزرسانی همه پارامترهای مدل داریم که هزینه حافظه و محاسباتی بالایی دارد.
- در روش **LoRA**، تصمیم می‌گیریم که وزن‌های اصلی مدل (مثلاً ماتریس‌های بزرگ لایه توجه) را ثابت نگه داریم و به جای آن در بخش‌های هدف (مثلاً لایه‌های q_proj , k_proj در self-attention) ماتریس‌هایی کم‌رتبه اضافه کنیم که فقط همین ماتریس‌ها طی یادگیری آپدیت شوند.
- این ماتریس‌های کم‌رتبه معمولاً از ابعاد $r \times fan_in$ و $fan_out \times r$ تشکیل می‌شوند؛ جایی که r همان رنک (Rank) است.

۱.۲. چرا «کم‌رتبه»؟

- ایده «کم‌رتبه» (low-rank) این است که تغییر مورد نیاز در وزن‌های بزرگ مدل، عموماً ساختاریافته و با بُعد موثر پایین‌تر از کل ابعاد آن لایه است. مثلاً اگر لایه‌ای ۶۵k پارامتر داشته باشد، می‌توان با یک ماتریس کوچک‌تر (رنک پایین) تغییر مطلوب را پیاده کرد.
- بدین ترتیب، فضایی که مدل می‌تواند «نسبت به وزن‌های پایه» حرکت کند محدودتر ولی کافی است تا دانش تازه در تسک جدید را کسب نماید.

۱.۳. جزئیات مزایا

۱. صرفه‌جویی در حافظه:

- در فاین‌تیون کامل، هر وزن مدل باید گرادیان، مومنوم و واریانس اوپتیمایزر (مثلاً در آدم) داشته باشد که حجم زیادی ایجاد می‌کند.
- در LoRA، به جای همه وزن‌ها، تنها آداپتورهایی با ابعاد کوچک + گرادیان + مومنوم + واریانس مربوط به آن‌ها ذخیره می‌شوند.

۲. پارامترهای کم و ساختار ماژولار:

- پس از یادگیری، دیگر نیازی نیست مدل را به‌طور کامل ذخیره کنیم. می‌توانیم همان وزن‌های پایه را نگه داریم + آداپتورهای کم‌حجم را به‌صورت افزونه (plug-in) داشته باشیم.

۳. سرعت:

- چون تعداد پارامترهای در حال به‌روزرسانی کوچک است، پس/انتشار (backprop) سریع‌تر انجام می‌شود و تبادل حافظه‌ی کمتری صورت می‌گیرد.

۱.۴. محدودیت‌ها و نکات طراحی

1. ظرفیت محدود:

- اگر رنک خیلی کوچک باشد یا ماژول‌های نامناسبی برای افزودن LoRA انتخاب شوند، شاید مدل نتواند همه پیچیدگی‌های تسک را بیاموزد (underfitting).
- به همین دلیل گاهی نیاز است رنک را بزرگ‌تر بگیریم یا ماژول‌های بیشتری را LoRA کنیم تا کیفیت بالا بماند.

2. طراحی نیازمند تجربه:

- بسته به مدل، گاهی فقط در ماژول‌های q_proj و v_proj LoRA قرار می‌دهند، گاهی همه پروجکشن‌های توجه یا حتی فیدفوروارد را پوشش می‌دهند. این تصمیم مستقیماً روی عملکرد و حجم نهایی تاثیر دارد.

۲. QLoRA Quantized LoRA

۲.۱. ایده اصلی در QLoRA

- در QLoRA، همان *آداپتورهای کم‌رتبه LoRA* را داریم، اما با این تفاوت که **وزن‌های پایه مدل** را به فرم ۴بیتی (Quantized) نگهداری می‌کنیم.
- بنابراین، اندازه‌ی حافظه اشغال‌شده توسط مدل اصلی بسیار کم می‌شود؛ ولی آداپتورهای LoRA همچنان با دقت بالاتر (مثلاً FP16 یا BF16) ذخیره و آپدیت می‌شوند.

۲.۲. چرا کوانتیزه‌ی ۴بیتی در پایه مدل؟

1. کاهش حافظه و هزینه سخت‌افزار:

- مثلاً اگر یک مدل ۱۰ میلیارد پارامتری را FP16 ذخیره کنید، نیاز به ۲۰ گیگابایت برای فقط وزن‌ها دارید. اما در QLoRA می‌توان همین را به یک‌چهارم (یا حتی کمتر) تقلیل داد.

2. ترکیب با LoRA:

- هرچند وزن‌ها کوانتیزه شده‌اند، اما یادگیری اصلی روی آداپتورهای کم‌رتبه است که دقت بالایی دارند و اجازه می‌دهند کیفیت حفظ شود.

۲.۳. محدودیت‌ها و ظرائف

- پیچیدگی کوانتیزاسیون:
اگر اجرای ۴ بیتی به درستی صورت نگیرد، ممکن است مدل پایه دچار افت دقت قابل توجه شود؛ به ویژه اگر نحوهٔ نرمال‌سازی یا نقشه‌برداری عددی اشتباه باشد.
- باقی ماندن محدودیت‌های LoRA:
در نهایت QLoRA هم دقیقاً مانند LoRA به انتخاب رنک، مازول‌ها و ... بستگی دارد. صرفاً مزیت بیشتری در کاهش حافظه فراهم می‌کند.

۳. LoRA Rank

۳.۱. تعریف Rank

- هنگامی که یک لایه مثلاً W دارد و ما می‌خواهیم با LoRA تغییرش دهیم، دو ماتریس کم‌رتبه اضافه می‌کنیم $\mathbf{A} \in \mathbb{R}^{d \times r}$ و $\mathbf{B} \in \mathbb{R}^{r \times d}$ (به صورت خلاصه). پارامتر rr همان Rank است.
- در عمل، وزنی به شکل $\mathbf{W} + \alpha \cdot \mathbf{AB}$ شکل می‌گیرد (که α معمولاً یک ضریب مقیاس کوچک است).

۳.۲. رنک بالا در مقابل رنک پایین

۱. رنک بالا:

- ماتریس کم‌رتبه $\mathbf{AB}$ فضای بزرگ‌تری را می‌پوشاند؛ بنابراین می‌تواند پیچیدگی‌های گوناگون‌تر تسک جدید را یاد بگیرد.
- اما پارامترهای اضافی بیشتری دارد (تقریباً $2 \times d \times r$) و حافظه و زمان بیشتری برای آپدیت لازم دارد.

۲. رنک پایین:

- سبک‌تر است. سریع‌تر تمرین می‌شود.
- اما اگر تسک جدید خیلی متفاوت یا پیچیده باشد، مدل با رنک پایین امکان دارد نتواند نمایشی دقیق از آن بسازد (افت دقت).

۳.۳. یافتن «رنک بهینه»

- یک اصل کلی این است که از عددهای متوسط (مانند ۸، ۱۶ یا ۳۲) استفاده می‌کنند و تجربه نشان داده برای بسیاری از تسک‌های زبانی کافی است.
 - گاهی نیاز به تجربه و ارزیابی روی مجموعه داده اعتبارسنجی است تا رنک مناسب انتخاب شود؛ توازن بین کیفیت و هزینه.
-

۴. دیتا تایپ NF4؛

۴.۱. تفاوت با int4 ساده

- در روش int4 ممکن است وزن‌ها به شکل خطی بین min و max نگاشت شوند؛ یا همه آن‌ها با یک مقیاس ثابت ذخیره شوند.
- در NF4 Normalized Float 4-bit، معمولاً رویکردی هوشمندتر به کار می‌رود:
 - برای هر بلاک کانال یا هر بلوک چند ده پارامتری، یک ضریب مقیاس (scale factor) محاسبه می‌شود تا آن بخش نرمال گردد.
 - سپس اعداد در محدوده ۴بیتی شناور ثبت می‌شوند.

۴.۲. مزیت NF4

- **دقت بهتر در توزیع وزن‌ها:**
چون دامنه عددی هر بلوک به شکل جداگانه تنظیم می‌شود، خطای کوانتیزه کردن برای محدوده‌های بزرگ یا کوچک کاهش می‌یابد.
- **حفظ ویژگی‌های مهم وزن‌ها:**
مثلاً اگر برخی وزن‌ها مثبت و برخی منفی باشند یا بردارهای مختلف دامنه‌های متفاوت داشته باشند، NF4 بهتر از int4 ساده عمل می‌کند.
- **کاهش پهنای باند:**
در حین پیش‌پردازش (forward pass)، خواندن وزن‌ها از حافظه سبک‌تر می‌شود و منابع محاسباتی آزادتر می‌گردد.

۴.۳. کاربرد در QLoRA

- QLoRA از NF4 برای نگهداری وزن‌های فریز شده‌ی مدل بهره می‌گیرد؛ یعنی مدل پایه را با NF4 ذخیره می‌کند.
- بعد در هنگام آموزش و استنتاج، این وزن‌ها به فرمت بالاتر (مثلاً FP16 داخلی) تبدیل می‌شوند تا محاسبات انجام گردد، ولی ذخیره‌سازی و انتقال در حافظه با فرمت ۴ بیتی است.

ماژول‌های مورد هدف در سوال (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj و down_proj) :

ماژول‌های مورد هدف در سوال (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj و down_proj) اجزای اصلی معماری مدل‌های مبتنی بر ترنسفورمر (مانند BERT، GPT یا LLaMA) هستند. این ماژول‌ها مسئول محاسبات و تغییراتی هستند که در مکانیزم توجه و لایه‌های پیش‌برنده (Feed-Forward) انجام می‌شود. در ادامه، هر یک را به‌طور کامل توضیح می‌دهیم:

1. ماژول‌های مکانیزم توجه (Self-Attention Mechanism)

مکانیزم توجه بخشی از مدل است که روابط بین توکن‌های یک جمله (یا توکن‌های ورودی) را محاسبه می‌کند. این ماژول‌ها شامل موارد زیر هستند:

q_proj (پروژکشن کوئری یا پرسش)

- هدف: ورودی‌های مدل (بردارهای تعبیه‌شده یا حالت‌های مخفی) را به فضای کوئری (پرسش) تبدیل می‌کند.
- نقش: مشخص می‌کند که هر توکن می‌خواهد به چه چیزی در توکن‌های دیگر توجه کند.
- فرمول ریاضی: $Q = XW_q$ که در آن XX ورودی و W_q ماتریس وزن کوئری است.

k_proj (پروژکشن کلید یا Key)

- هدف: ورودی‌ها را به فضای کلید تبدیل می‌کند.

- نقش: نشان می‌دهد که هر توکن چه اطلاعاتی برای دیگر توکن‌ها فراهم می‌کند تا به آن توجه کنند.
- فرمول ریاضی: $K = XW_kK = XW_k$ که در آن W_k ماتریس وزن کلید است.

v_proj (پروجکشن مقدار یا Value)

- هدف: ورودی‌ها را به فضای مقدار تبدیل می‌کند.
- نقش: اطلاعات واقعی را که پس از محاسبات توجه منتقل می‌شوند، شامل می‌شود.
- فرمول ریاضی: $V = XW_vV = XW_v$ که در آن W_v ماتریس وزن مقدار است.

o_proj (پروجکشن خروجی یا Output Projection)

- هدف: مقادیر وزنی (که نتیجه مکانیزم توجه هستند) را ترکیب کرده و به فضای ابعاد اولیه بازمی‌گردانند.
- نقش: اطمینان می‌دهد که خروجی مکانیزم توجه همان ابعاد ورودی را دارد.
- فرمول ریاضی: $O = AW_oO = AW_o$ که در آن AA مقادیر توجه‌شده و W_o ماتریس وزن خروجی است.

2. ماژول‌های لایه‌های پیش‌برنده (Feed-Forward Layers)

بعد از مکانیزم توجه، لایه‌های پیش‌برنده (FFN) برای پردازش و تبدیل خروجی مکانیزم توجه به کار می‌روند. این ماژول‌ها شامل موارد زیر هستند:

gate_proj (پروجکشن دروازه‌ای یا Gate Projection)

- هدف: کنترل جریان اطلاعات در لایه پیش‌برنده.
- نقش: مشابه مکانیزم‌های دروازه‌ای در شبکه‌های دیگر (مانند LSTM)، این ماژول قبل از پردازش اطلاعات در لایه، آن را تغییر شکل می‌دهد.

up_proj (پروجکشن صعودی یا Upward Projection)

- هدف: افزایش ابعاد داده به‌طور موقت در طول فرآیند پردازش.
- نقش: فضای ویژگی را گسترش می‌دهد تا امکان انجام محاسبات پیچیده‌تر فراهم شود.

- فرمول ریاضی:

داده‌ها از بعد اصلی dd به بعد بالاتر dupd_{up} منتقل می‌شوند.

down_proj (پروجکشن نزولی یا Downward Projection)

- هدف: کاهش ابعاد داده به مقدار اولیه پس از پروژکشن صعودی.
 - نقش: اطمینان می‌دهد که خروجی نهایی لایه پیش‌برنده همان ابعاد ورودی لایه ترنسفورمر را دارد.
 - فرمول ریاضی:
- داده‌ها از dupd_{up} به dd بازمی‌گردند.

چگونگی ارتباط این ماژول‌ها

1. در مکانیزم توجه:

- v_proj ، q_proj و k_proj برای محاسبه کوئری‌ها، کلیدها و مقادیر استفاده می‌شوند.
- این مقادیر برای محاسبه امتیاز توجه و تولید خروجی وزنی استفاده می‌شوند.
- نتیجه نهایی را به فضای اصلی بازمی‌گرداند.

2. در لایه‌های پیش‌برنده:

- $gate_proj$ اطلاعات را پیش از پردازش مدیریت می‌کند.
- up_proj ابعاد داده را افزایش داده و $down_proj$ آن را به مقدار اولیه بازمی‌گرداند.

چرا این ماژول‌ها هدف LoRA قرار می‌گیرند؟

- این ماژول‌ها شامل بزرگ‌ترین ماتریس‌های وزنی در مدل هستند و تعداد زیادی پارامتر دارند.
- تنظیم این ماژول‌ها تأثیر قابل‌توجهی بر عملکرد مدل دارد.
- روش LoRA فقط این ماتریس‌ها را تغییر داده و سایر پارامترهای مدل را ثابت نگه می‌دارد، که باعث کاهش نیاز به حافظه و زمان محاسباتی می‌شود.

با تمرکز روی این ماژول‌ها، LoRA به‌طور کارآمد مدل را با استفاده از تعداد کمی از پارامترهای قابل آموزش تنظیم می‌کند.

فرمول کلی تعداد پارامترها در هر ماژول بدین صورت محاسبه خواهند شد :

برای هر ماژول:

$$\{\text{تعداد پارامترها}\} = \{\text{ابعاد ورودی}\} * \{\text{ابعاد خروجی}\}$$

برای محاسبه تعداد پارامترهای مصرفی هر یک از ماژول‌های ذکرشده (q_proj , k_proj , v_proj , o_proj , up_proj , $gate_proj$ و $down_proj$)، باید به ابعاد ورودی و خروجی هر ماژول توجه کنیم. در اینجا، محاسبه تعداد پارامترها بر اساس معماری مدل‌های ترنسفورمری با ابعاد مشخص انجام می‌شود.

فرمول کلی تعداد پارامترها در هر ماژول

برای هر ماژول:

$$\text{ابعاد خروجی} \times \text{ابعاد ورودی} = \text{تعداد پارامترها}$$

اگر LoRA برای ماژول‌ها اعمال شود، ماتریس‌های A و B اضافه می‌شوند:

$$\text{LoRA (رتبه} \times \text{ابعاد اصلی)} \times 2 = \text{LoRA تعداد پارامترهای}$$

ماژول‌های مکانیزم توجه

1. q_proj, k_proj, v_proj

- این ماژول‌ها ابعاد ورودی را از فضای d (ابعاد مخفی) به همان فضای مخفی کاهش یا افزایش می‌دهند.
- تعداد پارامترها (بدون LoRA):

$$\text{تعداد پارامترها} = d \times d$$

2. o_proj

- این ماژول خروجی مکانیزم توجه را از فضای d به فضای مخفی برمی‌گرداند.
- تعداد پارامترها (بدون LoRA):

$$\text{تعداد پارامترها} = d \times d$$

ماژول‌های لایه‌های پیش‌برنده

3. gate_proj

- این ماژول معمولاً یک لایه پیش‌برنده است که اطلاعات را از فضای مخفی عبور می‌دهد.
- تعداد پارامترها (بدون LoRA):

$$\text{تعداد پارامترها} = d \times d_{\text{ffn}}$$

4. up_proj

- این ماژول ابعاد را از d به d_{ffn} افزایش می‌دهد.
- تعداد پارامترها (بدون LoRA):

$$\text{تعداد پارامترها} = d \times d_{\text{ffn}}$$

5. down_proj

- این ماژول ابعاد را از d_{ffn} به d کاهش می‌دهد.
- تعداد پارامترها (بدون LoRA):

$$\text{تعداد پارامترها} = d_{\text{ffn}} \times d$$

محاسبات برای LoRA

در LoRA، هر ماژول دو ماتریس A و B اضافه می‌کند:

$$\text{LoRA} = 2 \times (d \times r) \text{ تعداد پارامترهای}$$

مثال با مقادیر مشخص

فرض کنیم:

- $d = 4096$ (ابعاد فضای مخفی).

- $d_{\text{ffn}} = 4 \times d = 16384$ (ابعاد لایه پیش‌برنده).

- رتبه LoRA $r = 16$.

محاسبات برای هر ماژول

1. $q_proj, k_proj, v_proj, o_proj$:

- تعداد پارامترها (بدون LoRA):

$$4096 \times 4096 = 16,777,216$$

- تعداد پارامترهای LoRA:

$$2 \times (4096 \times 16) = 131,072$$

2. up_proj و $gate_proj$:

- تعداد پارامترها (بدون LoRA):

$$4096 \times 16384 = 67,108,864$$

- تعداد پارامترهای LoRA:

$$2 \times (4096 \times 16) = 131,072$$

3. $down_proj$:

- تعداد پارامترها (بدون LoRA):

$$16384 \times 4096 = 67,108,864$$

- تعداد پارامترهای LoRA:

$$2 \times (4096 \times 16) = 131,072$$

خلاصه تعداد پارامترها

ماژول	(پارامترها) LoRA بدون	(پارامترهای اضافه) LoRA با
q_proj	16,777,216	131,072
k_proj	16,777,216	131,072
v_proj	16,777,216	131,072
o_proj	16,777,216	131,072
gate_proj	67,108,864	131,072
up_proj	67,108,864	131,072
down_proj	67,108,864	131,072

در این پاسخ، تلاش می‌کنیم روش **CoPrompt** و شیوهٔ مقابلهٔ آن با فراموشی فاجعه‌بار (وقتی مدل پس از یادگیری یک وظیفه جدید، عملکرد کلی خود را از دست می‌دهد) را گام‌به‌گام و با جزئیات بیشتر توضیح دهیم تا کاملاً روشن شود چطور با ایجاد تغییرات ساختاری در مدل، این مشکل را کاهش می‌دهد و عملکرد مدل را در هر دو بُعد (یادگیری وظایف جدید و حفظ/افزایش توانایی عمومی) بهبود می‌بخشد.

۱. ایدهٔ کلی CoPrompt: حفظ انسجام بین مدل پیش‌تمرین‌شده و مدل فاین‌تیون‌شده

مدل‌های پیش‌تمرین‌شده (مثل CLIP) توانایی کلی و قدرت شناختی بالایی دارند زیرا روی دیتاست‌های بسیار بزرگ آموزش دیده‌اند. اگر ما برای یک تسک خاص (مثلاً تشخیص یک دسته تصاویر جدید) بخواهیم مدل را فاین‌تیون کنیم، معمولاً برخی از پارامترهای مدل را تغییر می‌دهیم یا پارامترهای جدیدی اضافه می‌کنیم. مشکل اینجاست که در داده‌های کم (few-shot)، مدل ممکن است «دانش» گستردهٔ خود را فدا کند و صرفاً روی آن داده‌ها بیش‌برازش کند که نتیجه‌اش فراموشی فاجعه‌بار است.

CoPrompt برای حل این مشکل، یک محدودیت انسجامی (Consistency Constraint) تعریف می‌کند که در واقع می‌گوید:

«پارامترهایی که برای یادگیری وظیفه جدید (few-shot) تغییر می‌کنند، نباید بازنمایی‌های نهایی مدل را زیاد از بازنمایی‌های قبلی (مدل اصلی) دور کنند.»

به عبارت ساده، خروجی مدل فاین‌تیون‌شده باید «شبهه» خروجی همان مدل پیش‌تمرین‌شده باقی بماند. در نتیجه:

1. توانایی یا دانش عمومی (general knowledge) مدل حفظ می‌شود.

2. از بیش‌برازش روی داده جدید جلوگیری می‌کند.

۲. واردکردن اغتشاش در ورودی‌ها برای آموزش بهتر (Perturbed Inputs)

۲.۱. متن

- در حالت عادی، برای نام کلاس مثلاً «cat»، مدل پیش‌تمرین‌شده CLIP از یک عبارت ساده مثل «a photo of a cat» استفاده می‌کند.
- در **CoPrompt**، برای ایجاد تنوع و جلوگیری از «یادگیری تک‌جمله‌ای»، از یک مدل زبان (مثلاً GPT) استفاده می‌شود تا یک جمله غنی‌تر بسازد (مثلاً «یک موجود چهارپای خردار با گوش‌های تیز»).
- سپس انسجام بین بردار این توصیف جدید و توصیف قدیمی بررسی و کنترل می‌شود.

۲.۲. تصویر

- از افزایش‌های تصویری (Image Augmentations) مثل برش تصادفی، چرخش، یا flip استفاده می‌شود تا چند تصویر تغییریافته از یک ورودی بسازیم.
- مدل باید خروجی (embedding) خود را برای این تصاویر اغتشاش‌یافته نزدیک به خروجی‌های مدل پیش‌تمرین‌شده نگه دارد.

این کار سبب می‌شود که مدل فاین‌تیون‌شده یاد بگیرد حتی اگر ورودی تصاویر/متن کمی عوض شد، همچنان به سبک «داده‌های گسترده» مدل اصلی عمل کند و از مسیر اصلی دور نشود.

۳. ترکیب دو روش مهم: پرامپت‌ها + آداپتورها

۳.۱. پرامپت‌ها (Prompts)

- پرامپت‌ها قطعاتی از بردارهای قابل‌آموزش هستند که در ورودی متن یا تصویر اضافه می‌شوند تا به مدل «راهنمایی» بیشتری دربارهٔ وظیفه جدید بدهند (مثلاً توضیح اضافی در بخش متنی یا ساختار جدید در ورودی تصویری).
- پیش از این روش‌ها، گاهی پرامپت‌ها فقط برای متن بودند یا فقط برای تصویر؛ برخی روش‌ها (مثل MaPLE) هر دو را داشتند اما ممکن بود مشکل بیش‌برازش به‌وجود بیاید.

۳.۲. آداپتورها (Adapters)

- آداپتورها ماژول‌های کوچکی هستند که در لایه‌های آخر مدل قرار می‌گیرند و بردار خروجی را تنظیم می‌کنند.
- مثلاً یک MLP دولایه که بردار ویژگی‌های مدل را می‌گیرد و خروجی را کمی دستکاری می‌کند تا به تسک جدید بهتر پاسخ دهد.

۳.۳. مزیت ترکیب

- **CoPrompt** می‌گوید: «چرا از هر دو روش با هم استفاده نکنیم؟». چون پرامپت‌ها در سطح ورودی (text) و (image) عمل می‌کنند، اما آداپتورها در سطح خروجی (feature) قرار دارند. بنابراین مجموعاً **انعطاف بیشتری** داریم.
- البته اضافه‌کردن پارامتر زیاد ممکن است خطر بیش‌برازش را افزایش دهد. اما محدودیت انسجامی (Consistency Constraint) کمک می‌کند این پارامترها همچنان در راستای مدل پیش‌تمرین‌شده باقی بمانند و از هدایت اشتباه جلوگیری می‌کند.

۴. چرا این کار جلو فراموشی فاجعه‌بار را می‌گیرد؟

1. **محدودیت انسجامی** مدل را وادار می‌کند که «دانش کلی» (General Knowledge) خود را کنار نگذارد و تنها در محدوده‌ای مشخص اجازه می‌دهد تا برای تسک جدید تنظیم شود.

2. **اغتشاش در ورودی** باعث می‌شود مدل مجبور شود روی یک طیف وسیع‌تری از جملات یا تصاویر خودش را «مطابق» کند. در نتیجه، صرفاً به داده‌های اندک بسنده نمی‌کند و قابلیت‌های پیشینش را حفظ می‌نماید.
3. **پرامپت‌ها + آداپتورها** می‌توانند آزادی عمل بیشتری برای یادگیری تسک جدید بدهند (بهتر فاین‌تیون شوند)، اما به خاطر قانون انسجام، نباید راه "غلط" را بروند. پس هم قدرت یادگیری بالا می‌رود و هم فراموشی رخ نمی‌دهد.

۵. دستاوردها و نتایج عملی

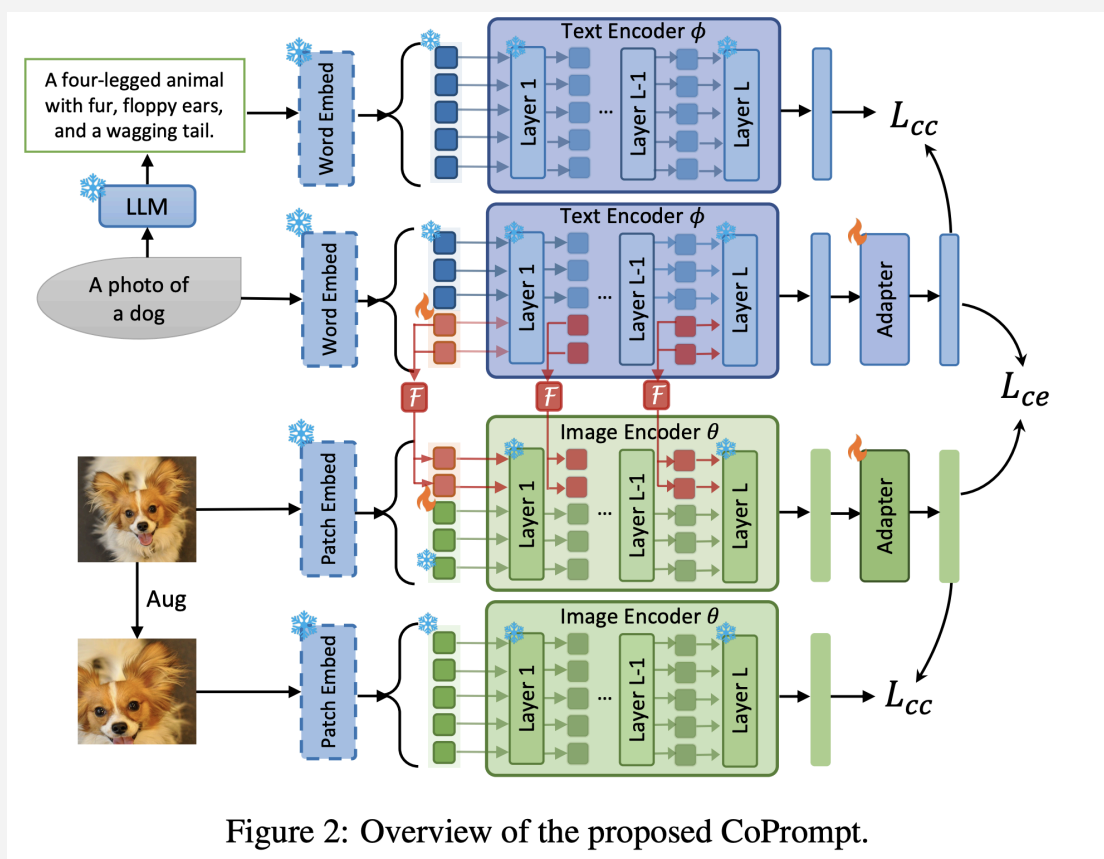
- طبق آزمایش‌های گزارش‌شده در مقاله:
 - **عملکرد مدل در وظیفه جدید (few-shot)** افزایش چشمگیری داشته است.
 - **توانایی zero-shot** (یعنی عمومی‌سازی مدل روی وظایف جدید بدون داده آموزشی) هم گاهی بیشتر از خود CLIP اولیه شده است؛ که بسیار جالب است چون معمولاً فاین‌تیونینگ سبب افت عملکرد zero-shot می‌شود.
 - در آزمون‌های cross-dataset (مدل روی یک دیتاست فاین‌تیون می‌شود ولی روی دیتاست دیگر تست می‌شود)، نتایج بسیار خوبی داشته است.

۶. جمع‌بندی کوتاه

CoPrompt:

1. **یک محدودیت انسجامی** تعریف می‌کند تا خروجی مدل فاین‌تیون‌شده نزدیک به خروجی مدل اصلی بماند.
2. از **ورودی‌های اغتشاش‌یافته** (جمله‌های توصیفی از LLM و تصاویر افزایشی) بهره می‌گیرد تا این انسجام محکم‌تر شود.
3. **پرامپت‌ها** (در ورودی متن و تصویر) و **آداپتورها** (در لایه‌های بالایی) را با هم ترکیب می‌کند؛ اما به کمک آن محدودیت انسجامی، مانع بیش‌برازش می‌شود.

نتیجه نهایی: مدل حتی پس از یادگیری تسک‌های خاص (با داده کم) همچنان دانش کلی خود را از دست نداده و می‌تواند در وظایف گسترده (zero-shot) عملکرد خوبی—or حتی بهتر—داشته باشد؛ یعنی مشکل فراموشی فاجعه‌بار حل می‌شود یا بسیار کاهش می‌یابد.



زیر بخش پنجم : پیاده سازی مدل ها

در ابتدا باید دیتاستی را که استفاده کرده ایم را معرفی کنیم :

گزارش و تحلیل دیتاست: [leonardPKU/clevr_cogen_a_train](https://huggingface.co/datasets/leonardPKU/clevr_cogen_a_train)

این دیتاست یک مجموعه داده چندوجهی (Multimodal) است که برای کارهای درک بصری و تولید زبان (Visual Question Answering - VQA) یا تولید توصیف زبان بر اساس تصویر (Visual-Language) طراحی شده است.

Generation طراحی شده است. این دیتاست از مجموعه داده‌ی اصلی **CLEVR** (Compositional Language and Elementary Visual Reasoning) مشتق شده و هدف آن آموزش مدل‌ها برای انجام استدلال‌های منطقی بر اساس محتوای بصری است.

مشخصات عمومی دیتاست بدین شکل می‌باشد :

مشخصه	مقدار/توضیح
شناسه (ID)	leonardPKU/clevr_cogen_a_train
وجه‌ها (Modalities)	تصویر (Image) و متن (Text)
اندازه کلی (Size)	بین ۱۰ هزار تا ۱۰۰ هزار سطر (طبق تصویر: 10K - 100K)
فرمت‌ها (Formats)	parquet ، Dask ، Croissant
Split موجود	train (شامل ۷۰ هزار سطر)
Export to Sheets 	

ساختار ستون‌های این دیتاست بدین شکل می‌باشد :

نام ستون	نوع داده/توضیح	مثال محتوایی	نقش در فرآیند آموزش
image	Image (عرض و ارتفاع ۴۸۰ پیکسل)		ورودی بصری مدل (Encoder Input)
problem	String (متن سؤال/مسئله)	How many items are there in the image	ورودی متنی مدل (Encoder Input)
solution	String (پاسخ صحیح به سؤال)	<answer> 4 </answer>	خروجی مورد انتظار/برچسب‌ها (Decoder Target/Labels)
Export to Sheets 			

۳. تحلیل محتوایی و وظیفه مدل

مدل PaliGemma که در این پروژه استفاده می‌شود، با این ساختار داده برای وظایف زیر تنظیم می‌شود:

- نوع کار: Visual Question Answering (VQA) (پاسخ به سؤالات بصری).
- سؤالات (**problem**): شامل سؤالاتی هستند که نیاز به استدلال فضایی و منطقی ساده بر اساس تصویر دارند (مثلاً شمارش اشیاء، تشخیص رنگ، شکل یا اندازه).
- فرمت پاسخ (**solution**): پاسخ‌ها در یک قالب ساختاریافته (غالباً با استفاده از تگ‌های `<answer>...</answer>`) ارائه شده‌اند. مدل باید یاد بگیرد که پاسخ صحیح را در این قالب خاص تولید کند.

۴. ارتباط با کد اجرا شده

داده‌های این دیتاست دقیقاً به روش زیر در کدهای قبلی استفاده شدند:

1. تابع `preprocess_function`:

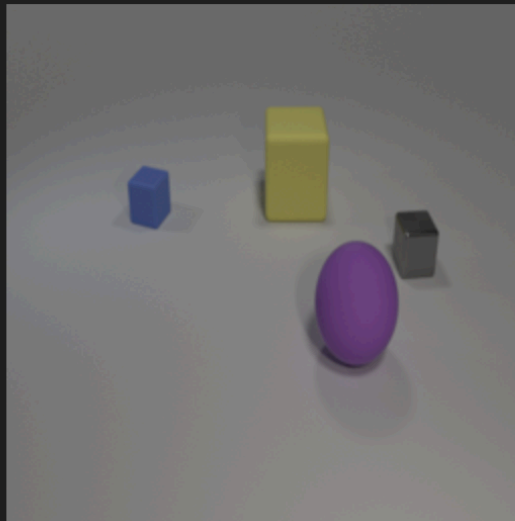
- تصاویر (`image`) را به اندازه ۲۲۴x۲۲۴ تغییر داد.
- سؤالات (`problem`) را با نشانه‌ی `<image>` ادغام کرد.
- پاسخ‌ها (`solution`) را توکنایز کرده و به عنوان برچسب (`Labels`) با مقادیر `-100` برای Padding آماده کرد.

2. فرآیند Fine-Tuning:

- مدل `PaliGemma` ورودی‌های بصری و متنی را دریافت می‌کند.
- هدف آموزش این است که مدل توالی توکن‌های پاسخ (`solution`) را به درستی، با توجه به محتوای تصویر و متن سؤال، پیش‌بینی کند.

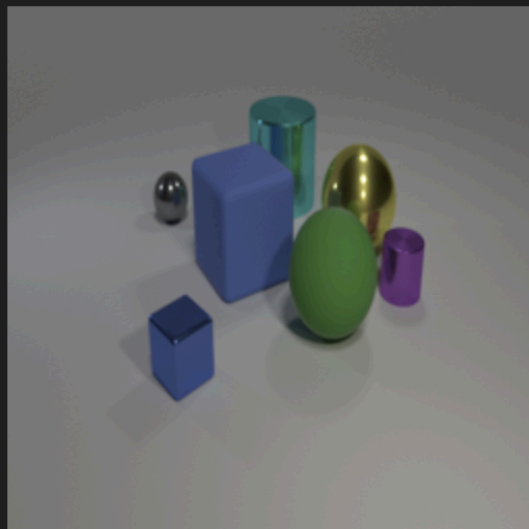
سمپل‌هایی از دیتاست :

--- Sample from Index: 2376 ---

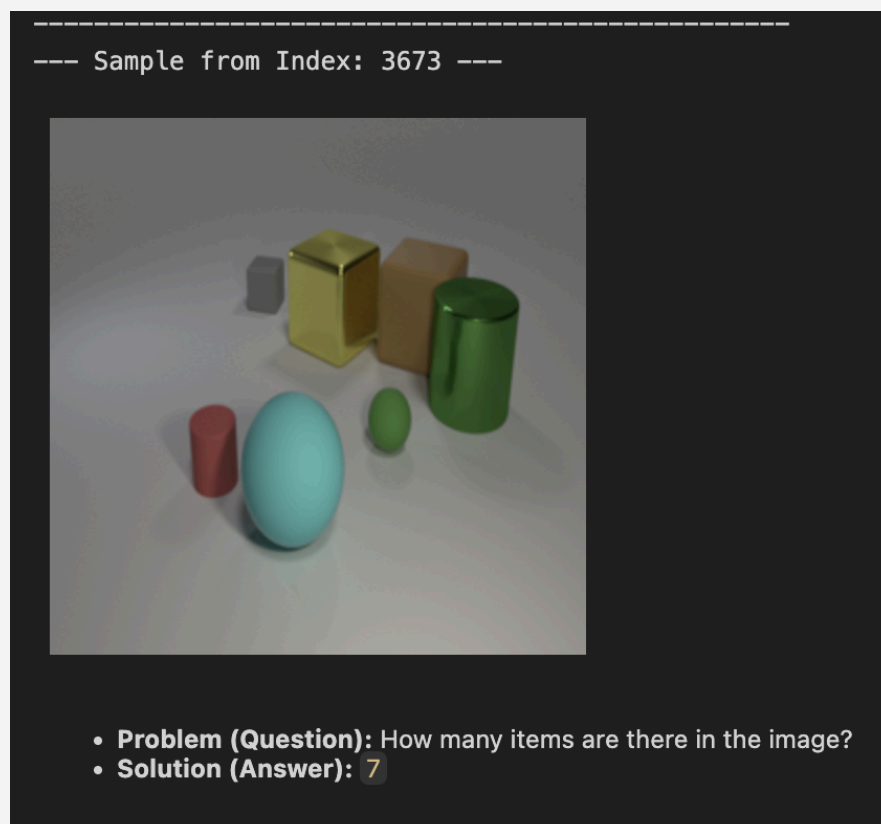


- **Problem (Question):** How many items are there in the image?
- **Solution (Answer):** 4

--- Sample from Index: 8169 ---



- **Problem (Question):** How many items are there in the image?
- **Solution (Answer):** 7



معرفی مدل paligemma-3b-mix-224:

پالی‌جما (PaliGemma-3b-mix-224) یک مدل زبان-بینایی (Vision-Language Model - VLM) پیشرفته و منبع باز است که توسط گوگل منتشر شده است. این مدل برای درک و پردازش همزمان تصاویر و متن طراحی شده و می‌تواند بر اساس ورودی‌های چندوجهی، خروجی متنی تولید کند.

google/paligemma-3b-mix-224 · like 85 · Following · Google · 30.6k

Image-Text-to-Text · Transformers · Safetensors · paligemma · image-to-text · text-generation-inference · arxiv20 papers · License: gemma

Model card · Files and versions · Community · Train · Deploy · Use this model

Gated model You have been granted access to this model

PaliGemma model card

Model page: [PaliGemma](#)

Transformers PaliGemma 3B weights, fine-tuned with 224*224 input images and 256 token input/output text sequences on a mixture of downstream academic datasets. The models are available in float32, bfloat16 and float16 format for research purposes only.

Resources and technical documentation:

- [Responsible Generative AI Toolkit](#)
- [PaliGemma on Kaggle](#)
- [PaliGemma on Vertex Model Garden](#)

Downloads last month: 530,765

Safetensors · Model size: 2.92B params · Tensor type: F32 · Files info

Inference Providers NEW

Image-Text-to-Text

This model isn't deployed by any Inference Provider. [Ask for provider support](#)

Model tree for google/paligemma-3b-mix-224

Adapters	19 models
Finetunes	21 models
Quantizations	1 model

۱. ساختار و معماری مدل

PaliGemma-3B-Mix-224 از ترکیب دو مؤلفه اصلی تشکیل شده است که با هم کار می‌کنند:

مؤلفه	مدل پایه	وظیفه اصلی
رمزگذار تصویر (Vision Encoder)	SigLIP (Sigmoid-based Large-scale Image-Text Pre-training)	تبدیل تصویر به توکن‌های بصری (Soft Tokens) قابل فهم برای مدل زبان.
رمزگشای زبان (Language Decoder)	Gemma-2B	دریافت توکن‌های بصری و متنی ادغام‌شده، و تولید متن (پاسخ یا شرح)

نحوه کار:

- پردازش تصویر:** رمزگذار SigLIP، تصویر ورودی (که در این نسخه دارای رزولوشن ۲۲۴x۲۲۴ است) را به بخش‌های کوچک (Patch) تقسیم کرده و ویژگی‌های بصری را به صورت یک دنباله از توکن‌های جاسازی‌شده (Embeddings) استخراج می‌کند.
- ادغام (Fusion):** این توکن‌های بصری به همراه توکن‌های متنی ورودی (مانند سؤال) به صورت یک دنباله واحد به رمزگشای Gemma داده می‌شوند.
- تولید متن:** مدل Gemma-2B بر اساس دنباله ادغام شده، به صورت خودرگرسیو (Autoregressive) کلمه به کلمه متن خروجی (پاسخ یا شرح) را تولید می‌کند.

۲. مشخصات اصلی و نامگذاری

- PaliGemma:** نشان‌دهنده خانواده مدل‌های زبان-بینایی مبتنی بر SigLIP و Gemma.
- 3B:** نشان‌دهنده تعداد تقریبی ۳ میلیارد پارامتر (وزن) مدل است. این حجم آن را در دسته مدل‌های با اندازه‌ی متوسط قرار می‌دهد که بین عملکرد خوب و کارایی منابع، تعادل ایجاد می‌کند.

- **mix**: این مدل از نوع **چک‌پوینت‌های ترکیبی (Mixed-task Checkpoints)** است. به این معنی که این مدل علاوه بر آموزش پیشین، بر روی مجموعه‌ای متنوع از وظایف بینایی-زبانی (مانند پاسخ به سؤال، کپشن‌نویسی، و استدلال فضایی) **تنظیم دقیق** شده است.
- **224**: نشان‌دهنده اندازه تصویر ورودی مورد انتظار مدل، یعنی **۲۲۴ در ۲۲۴ پیکسل** است.

۳. قابلیت‌ها و کاربردها

PaliGemma-3B-Mix-224 به دلیل آموزش دیدن بر روی مجموعه‌ای از وظایف ترکیبی، قابلیت‌های گسترده‌ای در زمینه بینایی-زبان دارد:

- **پاسخ به سؤال بصری (VQA)**: توانایی پاسخ به سؤالات پیچیده یا ساده درباره محتوای تصویر (مانند مثال‌های دیتاست CLEVR).
- **تولید کپشن (Image Captioning)**: شرح دادن محتوای تصویر به صورت کوتاه یا مفصل.
- **استدلال بصری (Visual Reasoning)**: درک روابط بین اشیاء در تصویر و استنتاج منطقی (مانند سؤالات شمارشی و مقایسه‌ای).
- **تشخیص متن (OCR)**: توانایی خواندن و بازتولید متن موجود در تصاویر.
- **انتقال یادگیری (Transfer Learning)**: به دلیل معماری باز و انعطاف‌پذیر، این مدل یک پایه عالی برای **تنظیم دقیق (Fine-Tuning)** بر روی وظایف خاص در دامنه دلخواه شما (همانطور که در کدهای قبلی انجام شد) محسوب می‌شود.

خلاصه: این مدل یک ابزار قدرتمند و بهینه شده برای شروع کار در حوزه بینایی-زبان است که با حجم پارامتر مناسب، امکان آموزش کارآمد (مخصوصاً با تکنیک LoRA) بر روی سخت‌افزارهای استانداردتر را فراهم می‌کند.

در این قسمت هر بخش از کد ای را که داریم را توضیح می‌دهیم :

بخش اول : Environment Setup and Dataset Loading

```

!pip install -q peft transformers datasets evaluate bitsandbytes rouge_score huggingface_hub
import os
import numpy as np
import logging
from PIL import Image
import torch
import random
import json

from datasets import load_dataset
from peft import LoraConfig, get_peft_model
from transformers import (
    PaliGemmaProcessor,
    PaliGemmaForConditionalGeneration,
    Trainer,
    TrainingArguments,
    BitsAndBytesConfig,
)
import evaluate
from huggingface_hub import notebook_login

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

if torch.cuda.is_available():
    device = torch.device("cuda")
    logger.info(f"Using device: {torch.cuda.get_device_name(0)}")
else:
    device = torch.device("cpu")
    logger.info("GPU not available, using CPU instead.")

logger.info("Loading the clevr_cogen_a_train dataset...")

full_subset = load_dataset("leonardPKU/clevr_cogen_a_train", split="train[:20%]")
split_datasets = full_subset.train_test_split(test_size=0.1, seed=42)

train_dataset = split_datasets["train"]
test_dataset = split_datasets["test"]

logger.info(f"Training dataset size: {len(train_dataset)}")
logger.info(f"Testing dataset size: {len(test_dataset)}")

```

1. نصب وابستگی‌ها: خط اول کتابخانه‌های مورد نیاز مانند `peft`، `transformers`، `datasets`،

`rouge_score` و `bitsandbytes` را برای آموزش مدل نصب می‌کند.

2. وارد کردن کتابخانه‌ها: ماژول‌های لازم برای کار با مدل‌های ترانسفورمرز، داده‌ها، تصاویر، و PyTorch و

تنظیم دقیق (PEFT) وارد می‌شوند.

3. **تنظیم دستگاه:** کد، دستگاه محاسباتی را بررسی کرده و **GPU (CUDA)** را در صورت دسترسی یا **CPU** را در غیر این صورت برای اجرای سریع‌تر مدل انتخاب می‌کند.
4. **بارگیری داده:** دیتاست **"leonardPKU/clevr_cogen_a_train"** بارگیری می‌شود، اما فقط ۲۰٪ از مجموعه آموزشی اصلی (برای سرعت).
5. **تقسیم داده:** داده‌های بارگیری شده به دو بخش **آموزش (۹۰٪)** و **آزمایش (۱۰٪)** تقسیم می‌شوند تا عملکرد مدل ارزیابی شود.
6. **مدل و پردازشگر:** این کد آمادگی لازم برای بارگیری مدل از پیش‌آموزش دیده **PaliGemma** و **PaliGemmaProcessor** مربوطه را فراهم می‌کند (اگرچه خطوط بارگیری خود مدل در قسمت نمایش داده شده نیست).
7. **کوانتیزاسیون (Quantization):** برای کاهش مصرف حافظه، از **BitsAndBytesConfig** برای بارگیری مدل در دقت ۴ بیت استفاده خواهد شد (تکنیک معمول در LoRA).
8. **تنظیم LoRA:** یک **LoraConfig** برای تعریف پارامترهای LoRA (مثل **r** و **lora_alpha**) تعریف می‌شود تا فقط لایه‌های کمی از مدل در طول آموزش به‌روز شوند.
9. **آماده‌سازی مدل PEFT:** مدل **PaliGemma** با استفاده از **get_peft_model** برای پذیرش تنظیمات LoRA آماده‌سازی می‌شود.
10. **آموزش و ارزیابی:** در نهایت، **TrainingArguments** و **Trainer** برای شروع فرآیند تنظیم دقیق (آموزش با داده‌های آموزش و ارزیابی با داده‌های آزمایش) تعریف و آماده می‌شوند.

بخش دوم : Model and LoRA Configuration

```
model_id = "./paligemma-3b-mix-224"
logger.info(f"Loading processor from {model_id}...")

processor = PaliGemmaProcessor.from_pretrained(model_id)

bnb_config = BitsAndBytesConfig(
    load_in_8bit=True,
    llm_int8_threshold=6.0,
)

logger.info("Loading PaliGemma model in 8-bit precision...")

model = PaliGemmaForConditionalGeneration.from_pretrained(
```

```

model_id,
device_map="auto",
quantization_config=bnb_config,
)

logger.info("Configuring LoRA for efficient fine-tuning...")

lora_config = LoraConfig(
    r=64,
    lora_alpha=64,
    lora_dropout=0.05,
    target_modules=[
        "q_proj",
        "o_proj",
        "k_proj",
        "v_proj",
        "gate_proj",
        "up_proj",
        "down_proj",
    ],
    task_type="CAUSAL_LM",
)

model = get_peft_model(model, lora_config)

logger.info("Trainable parameters after applying LoRA:")

model.print_trainable_parameters()

```

هدف: آماده‌سازی PaliGemma برای Fine-Tuning کارآمد

این بخش از کد، مراحل حیاتی برای بارگیری، بهینه‌سازی حافظه و پیکربندی یک مدل بزرگ بصری-زبانی (VLM) به نام **PaliGemma-3B** را برای تنظیم دقیق (Fine-Tuning) با استفاده از تکنیک **LoRA** (Low-Rank Adaptation) پوشش می‌دهد.

۱. بارگیری پردازشگر (Processor)

اولین گام، بارگیری **Pali Gemma Processor** است. این ابزار نقش مترجم را بازی می‌کند:

- تصاویر را آماده می‌کند: اندازه آنها را تنظیم و نرمال‌سازی می‌کند تا مدل بتواند آنها را درک کند.
- متن را آماده می‌کند: جملات را به توکن (Token) یا کلماتی که مدل می‌فهمد، تبدیل می‌کند (توکنایز کردن).

۲. بارگیری مدل با کوانتیزاسیون (Quantization)

مدل **PaliGemma** به دلیل داشتن تقریباً ۳ میلیارد پارامتر (وزن) بسیار بزرگ است. برای جلوگیری از پُر شدن حافظه GPU، از تکنیک کوانتیزاسیون ۸-بیتی (**8-bit Quantization**) استفاده می‌کنیم:

- **BitsAndBytesConfig**: این تنظیمات را تعریف می‌کند که مدل به جای دقت استاندارد ۳۲-بیتی، در دقت ۸-بیتی بارگیری شود.
- این کار مصرف حافظه را چهار برابر کاهش می‌دهد و امکان آموزش روی کارت‌های گرافیک استانداردتر را فراهم می‌کند.

۳. پیکربندی و اعمال (LoRA (Low-Rank Adaptation

هدف ما این نیست که تمام ۳ میلیارد پارامتر مدل را دوباره آموزش دهیم (که بسیار زمان‌بر و نیازمند منابع زیاد است). به جای آن، از **LoRA** استفاده می‌کنیم:

- **LoRA Config**: یک سری ماتریس‌های کوچک (آداپتور) را در لایه‌های کلیدی مدل (مانند **q_proj** و **v_proj**, **gate_proj**) تزریق می‌کند.
- **r=64**: این پارامتر (Rank) قدرت این آداپتورهای کوچک را کنترل می‌کند.
- **get_peft_model(model, lora_config)**: با اعمال این تنظیمات، مدل اصلی "فریز" (قفل) می‌شود و فقط این آداپتورهای کوچک LoRA (که تنها ۱.۵۴٪ از کل پارامترها هستند) در طول آموزش به‌روزرسانی می‌شوند.

نتیجه: مدل، دانش اصلی خود را حفظ می‌کند اما رفتار خود را بر اساس داده‌های جدید تنظیم می‌کند، در حالی که سریع‌تر آموزش می‌بیند و از منابع کمتری استفاده می‌کند.

```
def preprocess_function(batch):
    questions = batch["problem"]
    images = batch["image"]
    answers = batch["solution"]

    processed_images = []
```

```

texts_with_image = []

for q, img in zip(questions, images):
    try:
        pil_img = img.convert("RGB").resize((224, 224))
        processed_images.append(pil_img)
        texts_with_image.append("<image> " + q)
    except Exception as e:
        logger.warning(f"Error processing an image, skipping it: {e}")
        processed_images.append(None)
        texts_with_image.append(None)

valid_indices = [i for i, img in enumerate(processed_images) if img is not None]
if not valid_indices:
    return {}

processed_images = [processed_images[i] for i in valid_indices]
texts_with_image = [texts_with_image[i] for i in valid_indices]
valid_answers = [answers[i] for i in valid_indices]

encoder_inputs = processor(
    images=processed_images,
    text=texts_with_image,
    padding="max_length",
    truncation=True,
    max_length=300,
    return_tensors="pt",
)

decoder_inputs = processor.tokenizer(
    text_target=valid_answers,
    padding="max_length",
    truncation=True,
    max_length=300,
    return_tensors="pt",
)

labels_ids = decoder_inputs["input_ids"].clone()
padding_mask = decoder_inputs["attention_mask"] == 0
labels_ids[padding_mask] = -100
encoder_inputs["labels"] = labels_ids
return encoder_inputs

logger.info("Applying preprocessing to the training dataset...")
processed_train_dataset = train_dataset.map(
    preprocess_function, batched=True, remove_columns=train_dataset.column_names
)

logger.info("Applying preprocessing to the testing dataset...")
processed_test_dataset = test_dataset.map(
    preprocess_function, batched=True, remove_columns=test_dataset.column_names
)

```

توضیح تابع `preprocess_function`

این تابع نحوه تبدیل یک دسته (Batch) از داده‌های خام (شامل تصویر، سؤال و جواب) را به تنسورهای ورودی و خروجی مناسب برای آموزش مدل تعیین می‌کند.

۱. پردازش تصاویر و فرمت‌بندی متن (Visual and Text Formatting)

تابع در هر دسته داده، کارهای زیر را به ازای هر نمونه (Sample) انجام می‌دهد:

- **تصویر:** شیء تصویر (Image Object) را می‌گیرد، آن را به فرمت **RGB** تبدیل می‌کند و سپس اندازه آن را به **224x224 پیکسل** (اندازه مورد انتظار PaliGemma) تغییر می‌دهد.
- **متن ورودی (Input Text):** متن سؤال (**problem**) را می‌گیرد و نشانه‌ی **<image>** را در ابتدای آن قرار می‌دهد (مثلاً: **<image> What color is the sphere?**). این نشان‌دهنده این است که تصویر باید توسط مدل قبل از خواندن سؤال پردازش شود.
- **حذف نمونه‌های نامعتبر:** اگر پردازش تصویری با خطا مواجه شود، آن نمونه را نادیده می‌گیرد تا فرآیند آموزش متوقف نشود.

۲. توکنایز کردن ورودی مدل (Encoder Inputs)

برای آماده‌سازی ورودی، توابع زیر از **processor** مدل استفاده می‌شوند:

- **ورودی‌ها:** شامل لیست تصاویر پردازش‌شده (**processed_images**) و لیست متون فرمت‌بندی‌شده (**texts_with_image**) است.
- **پدینگ و برش (Padding/Truncation):** توکن‌ها را تا طول **۳۰۰** پد (Padding) می‌کند یا برش می‌دهد (Truncation) تا همه ورودی‌ها اندازه یکسانی داشته باشند.
- **خروجی:** تنسورهای **input_ids** (توکن‌های ورودی) و **attention_mask** (که به مدل می‌گوید کدام توکن‌ها Padding هستند) را برای بخش رمزگذار (**Encoder**) مدل ایجاد می‌کند.

۳. توکنایز کردن برچسب‌ها (Labels/Targets)

PaliGemma از معماری رمزگذار-رمزگشا (Encoder-Decoder) استفاده می‌کند، بنابراین ما باید خروجی مورد انتظار (جواب) را نیز به‌طور جداگانه آماده کنیم:

- **ورودی‌ها:** لیست جواب‌های صحیح (`valid_answers`) توکنایز می‌شوند.
 - **خروجی:** تنسورهای `input_ids` و `attention_mask` برای بخش رمزگشا (Decoder) ایجاد می‌شوند. این توکن‌های خروجی هدف ما هستند و در نهایت به عنوان برچسب‌ها (`labels`) به مدل داده می‌شوند.
-

۴. آماده‌سازی نهایی برچسب‌ها (Final Label Preparation)

برای اینکه مدل فقط توکن‌های واقعی جواب را آموزش ببیند و توکن‌های Padding را نادیده بگیرد، یک گام حیاتی انجام می‌شود:

- **جایگزینی Padding:** توکن‌های Padding در برچسب‌ها (که در `attention_mask` مشخص شده‌اند) با مقدار ویژه `-100` جایگزین می‌شوند.
 - **نقش `-100`:** در کتابخانه Hugging Face، مقدار `-100` به تابع زیان (Loss Function) مدل علامت می‌دهد که نباید ضرر (Loss) را برای آن توکن خاص محاسبه کند؛ این یعنی توکن‌های اضافی Padding در فرآیند آموزش تأثیری نخواهند داشت.
-

کاربرد تابع (Mapping the Datasets)

در خطوط پایانی کد، تابع `preprocess_function` بر روی مجموعه داده‌های خام اعمال می‌شود:

- `train_dataset.map(...)`: تابع روی کل مجموعه داده‌های آموزشی و آزمایشی اجرا می‌شود.
- `batched=True`: این تضمین می‌کند که تابع یک دسته از داده‌ها را در یک زمان پردازش می‌کند که این کار فرآیند را بسیار سریع‌تر می‌کند.

- **remove_columns**: ستون‌های اصلی و خام (مانند **problem** و **image**) حذف می‌شوند، زیرا ما اکنون داده‌های پردازش‌شده (تنسورها) را داریم که آماده تغذیه به مدل هستند.

بخش چهارم: Trainer Setup and Fine-Tuning

```
training_args = TrainingArguments(
    output_dir="./paligemma-clevr-finetuned",
    num_train_epochs=1,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=4,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="steps",
    save_steps=100,
    logging_steps=10,
    learning_rate=1e-4,
    load_best_model_at_end=True,
    report_to="none",
    remove_unused_columns=False,
    fp16=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=processed_train_dataset,
    eval_dataset=processed_test_dataset,
    tokenizer=processor.tokenizer,
)

logger.info("Starting the fine-tuning process...")

trainer.train()
logger.info("Fine-tuning completed.")
```

۱. تعریف آرگومان‌های آموزش (TrainingArguments)

کلاس **TrainingArguments** تمام تنظیمات و فرایپارامترهای (Hyperparameters) مورد نیاز برای کنترل نحوه اجرای آموزش توسط **Trainer** را مشخص می‌کند:

- **مسیر خروجی (output_dir)**: مدل‌های ذخیره‌شده (Checkpoints) و لاگ‌های آموزش در پوشه **paligemma-clevr-finetuned/** ذخیره خواهند شد.

- تعداد دوره‌های آموزش (`num_train_epochs`): مدل فقط برای ۱ دور (Epoch) بر روی کل مجموعه داده‌های آموزشی اجرا می‌شود.
- اندازه دسته‌ای (Batch Size):
 - `per_device_train_batch_size=4`: در هر مرحله، ۴ نمونه داده بر روی هر GPU برای آموزش استفاده می‌شود.
 - `per_device_eval_batch_size=4`: برای ارزیابی (Evaluation) نیز از اندازه‌ی دسته‌ای ۴ استفاده می‌شود.
- تجمع گرادینان (`gradient_accumulation_steps`): گرادینان‌ها در ۴ گام (Step) تجمع می‌شوند. این امر باعث می‌شود که اندازه دسته‌ای مؤثر (Effective Batch Size) به $16 = 4 \times 4$ برسد، که آموزش را پایدارتر می‌کند.
- استراتژی‌های ذخیره‌سازی و ارزیابی:
 - `eval_strategy="steps"` و `eval_steps=100`: مدل هر ۱۰۰ گام ارزیابی می‌شود.
 - `save_strategy="steps"` و `save_steps=100`: مدل هر ۱۰۰ گام در مسیر خروجی ذخیره می‌شود.
 - `load_best_model_at_end=True`: پس از اتمام آموزش، بهترین مدل (بر اساس نتایج ارزیابی) بارگیری و حفظ خواهد شد.
- نرخ یادگیری (`learning_rate`): مقدار 4-10 برای تنظیم میزان تغییر وزن‌ها در هر گام انتخاب شده است.
- دقت محاسباتی (`fp16`): استفاده از دقت ۱۶-بیتی (Half-precision) برای افزایش سرعت آموزش و کاهش مصرف حافظه GPU.

۲. راه‌اندازی کلاس Trainer

کلاس `Trainer`، هسته‌ی اصلی فرآیند آموزش در کتابخانه `transformers` است. این کلاس با اتصال مدل، داده‌ها و تنظیمات، فرآیند را سازماندهی می‌کند:

- `model=model`: مدل `PaliGemma` که قبلاً با کوانتیزاسیون ۸-بیتی و LoRA پیکربندی شده بود، به `Trainer` داده می‌شود.
- `args=training_args`: تنظیمات تعریف‌شده در مرحله قبل اعمال می‌شوند.
- `train_dataset` و `eval_dataset`: مجموعه داده‌های آموزشی و آزمایشی که در مرحله قبل با موفقیت پیش‌پردازش شده بودند، به `Trainer` اختصاص داده می‌شوند.
- `tokenizer=processor.tokenizer`: توکنایزر مدل برای مدیریت ورودی و خروجی‌های متنی در طول آموزش و ارزیابی مشخص می‌شود.

۳. اجرای آموزش (Training Execution)

- `trainer.train()`: با فراخوانی این متد، فرآیند تنظیم دقیق آغاز می‌شود. Trainer از تنظیمات و داده‌های ارائه شده استفاده می‌کند تا مدل را بر روی دیتاسِت CLEVR-CoGen-A آموزش دهد.
- در طول این فرآیند، Trainer به صورت دوره‌ای مدل را ارزیابی، لاگ‌ها را ثبت و مدل‌های میانی را ذخیره می‌کند (بر اساس `eval_steps` و `save_steps`).
- پس از تکمیل ۱ دوره، آموزش متوقف شده و به دلیل تنظیم `load_best_model_at_end=True`، بهترین نسخه‌ی مدل برای استفاده‌های بعدی بارگیری می‌شود.

بخش پنجم : Full Evaluation and Saving Results

```
from tqdm import tqdm

logger.info("Starting full evaluation on the test set...")

EVAL_IMAGE_DIR = "evaluation_images"
os.makedirs(EVAL_IMAGE_DIR, exist_ok=True)
logger.info(f"Evaluation images will be saved in: {EVAL_IMAGE_DIR}")

rouge_metric = evaluate.load("rouge")

eval_model = trainer.model
eval_model.eval()
eval_model.to(device)

evaluation_results = []

with torch.no_grad():
    for i in tqdm(
        range(len(processed_test_dataset)), desc="Evaluating and Saving Images"
    ):
        processed_sample = processed_test_dataset[i]
        original_sample = test_dataset[i]

        image_path = os.path.join(EVAL_IMAGE_DIR, f"eval_image_{i}.png")
        original_sample["image"].save(image_path)

        pixel_values = (
```

```

        torch.tensor(processed_sample["pixel_values"]).unsqueeze(0).to(device)
    )
    input_ids = torch.tensor(processed_sample["input_ids"]).unsqueeze(0).to(device)
    labels = processed_sample["labels"]

    outputs = eval_model.generate(
        pixel_values=pixel_values,
        input_ids=input_ids,
        max_length=300,
        early_stopping=True,
    )

    prediction_text = processor.tokenizer.decode(
        outputs[0], skip_special_tokens=True
    )

    labels[labels == -100] = processor.tokenizer.pad_token_id
    label_text = processor.tokenizer.decode(labels, skip_special_tokens=True)

    evaluation_results.append(
        {
            "image_path": image_path,
            "question": original_sample["problem"],
            "prediction": prediction_text,
            "ground_truth": label_text,
        }
    )

all_preds = [res["prediction"] for res in evaluation_results]
all_labels = [res["ground_truth"] for res in evaluation_results]

rouge_results = rouge_metric.compute(predictions=all_preds, references=all_labels)

print("\n--- Quantitative Evaluation Results ---")
print(f"ROUGE-1: {rouge_results['rouge1']:.4f}")
print(f"ROUGE-2: {rouge_results['rouge2']:.4f}")
print(f"ROUGE-L: {rouge_results['rougeL']:.4f}")
print("-----\n")

output_eval_file = "full_evaluation_results.json"
with open(output_eval_file, "w") as f:
    json.dump(evaluation_results, f, indent=4)
logger.info(f"Full evaluation results saved to {output_eval_file}")

print("\n--- Full Qualitative Evaluation Log ---")
for result in evaluation_results:
    print(f"\n--- Image: {result['image_path']} ---")
    print(f"Question: {result['question']}")
    print(f"Ground Truth: {result['ground_truth']}")

```

```
print(f"Prediction:  {result['prediction']}")
print("-" * (len(result["image_path"]) + 14))
```

۱. آماده‌سازی و بارگیری ابزارها

- **نوار پیشرفت (tqdm):** برای نمایش بصری پیشرفت حلقه ارزیابی استفاده می‌شود.
- **پوشه ذخیره تصاویر (EVAL_IMAGE_DIR):** یک پوشه به نام `evaluation_images` ایجاد می‌شود تا تصاویر مورد استفاده در هر نمونه ارزیابی در آنجا ذخیره شوند. این کار به تحلیل‌گر اجازه می‌دهد تا عملکرد مدل را در کنار ورودی بصری مربوطه بررسی کند.
- **بارگیری معیار ROUGE:** معیار `ROUGE` (Recall-Oriented Understudy for Gisting Evaluation) از کتابخانه `evaluate` بارگیری می‌شود. این معیار به طور معمول برای سنجش کیفیت خلاصه‌سازی و تولید متن استفاده می‌شود و شباهت بین متن تولیدشده توسط مدل (پیش‌بینی) و متن واقعی (حقیقت زمینی) را اندازه‌گیری می‌کند.
- **آماده‌سازی مدل:**
 - `eval_model = trainer.model`: مدل نهایی (که بهترین عملکرد را در طول آموزش داشته است) از `trainer` بارگیری می‌شود.
 - `eval_model.eval()` و `eval_model.to(device)`: مدل به حالت ارزیابی (Evaluation mode) درآمده و به دستگاه محاسباتی (GPU) منتقل می‌شود.
- **evaluation_results:** یک لیست خالی برای ذخیره نتایج ارزیابی کیفی هر نمونه.

۲. حلقه ارزیابی کامل (Full Evaluation Loop)

حلقه ارزیابی بر روی تمام نمونه‌های موجود در `processed_test_dataset` اجرا می‌شود:

- **جلوگیری از گرادیان (with torch.no_grad()):** این بلوک تضمین می‌کند که هیچ گرادیانی در طول ارزیابی محاسبه نمی‌شود، که منجر به کاهش مصرف حافظه و تسریع پردازش می‌شود.
- **ذخیره تصویر:** تصویر اصلی (`original_sample["image"]`) برای هر نمونه در پوشه `evaluation_images` ذخیره می‌شود.
- **آماده‌سازی ورودی:** تنسورهای `pixel_values` (تصویر) و `input_ids` (سؤال) از داده‌های پیش‌پردازش‌شده استخراج شده و به دستگاه (GPU) منتقل می‌شوند.

- تولید پاسخ (`eval_model.generate(...)`):
 - مدل با استفاده از ورودی‌های بصری و متنی، پاسخ را تولید می‌کند.
 - `max_length=300` و `early_stopping=True` پارامترهای تولید متن را کنترل می‌کنند.
 - تبدیل خروجی به متن:
 - `prediction_text`: خروجی عددی مدل (`outputs`) به یک رشته متنی قابل خواندن توسط انسان تبدیل می‌شود.
 - `label_text`: برچسب‌های حقیقت زمینی (`labels`) پس از جایگزینی توکن‌های -100 با توکن Padding، به رشته متنی تبدیل می‌شوند تا با متن پیش‌بینی مقایسه شوند.
 - ذخیره نتایج کیفی: یک دیکشنری حاوی مسیر تصویر، سؤال، پاسخ پیش‌بینی‌شده و حقیقت زمینی برای هر نمونه به لیست `evaluation_results` اضافه می‌شود.
-

۳. تحلیل کمی با ROUGE و ذخیره نتایج

پس از اتمام حلقه، نتایج ارزیابی کمی و کیفی نهایی می‌شوند:

- محاسبه ROUGE: معیار `rouge_metric` با استفاده از لیست کامل پیش‌بینی‌ها و برچسب‌های حقیقت زمینی محاسبه می‌شود. مقادیر ROUGE-1، ROUGE-2 و ROUGE-L که معیارهای اصلی شباهت متن هستند، چاپ می‌شوند.
- ذخیره نتایج کیفی: لیست کامل `evaluation_results` (شامل جزئیات هر نمونه) در یک فایل JSON با نام `full_evaluation_results.json` ذخیره می‌شود تا امکان تحلیل عمیق‌تر وجود داشته باشد.
- چاپ گزارش کیفی: یک حلقه نهایی، نتایج کیفی را به صورت منظم چاپ می‌کند، که یک بازخورد فوری در مورد چگونگی عملکرد مدل برای نمونه‌های خاص ارائه می‌دهد.

معیارهای ارزیابی:

معیار ارزیابی کمی: ROUGE Score

ROUGE (مخفف Recall-Oriented Understudy for Gisting Evaluation) یک مجموعه از معیارها است که برای ارزیابی خودکار کیفیت متنی که توسط ماشین تولید شده (پیش‌بینی مدل) در مقایسه با متون مرجع انسانی (حقیقت زمینی یا پاسخ صحیح) به کار می‌رود.

در این کد، از سه زیرمجموعه اصلی ROUGE برای سنجش مدل (Visual Question Answering (VQA استفاده شده است.

۱. ROUGE-N

ROUGE-N بر اساس مفهوم **n-gram** عمل می‌کند. N-gram به توالی‌های متوالی N کلمه‌ای اشاره دارد. این معیار همپوشانی (Overlap) توالی‌های N-کلمه‌ای بین پاسخ تولیدی مدل و پاسخ صحیح را می‌سنجد.

• ROUGE-1 (Unigram Overlap):

- این معیار همپوشانی **تک‌کلمه‌ای (Unigram)** را می‌سنجد. به عبارت دیگر، تعداد کلمات مجزایی را که در هر دو متن وجود دارند، محاسبه می‌کند.
- اهمیت:** نشان‌دهنده میزان **صحت محتوایی** پاسخ است، یعنی چقدر کلمات کلیدی و مهم در پاسخ مدل آمده‌اند.

• ROUGE-2 (Bigram Overlap):

- این معیار همپوشانی **دوکلمه‌ای متوالی (Bigram)** را اندازه‌گیری می‌کند.
- اهمیت:** ارزیابی می‌کند که آیا مدل توانسته است **توالی‌های کوچک کلمات** را به درستی حفظ کند یا خیر. این معیار به صورت غیرمستقیم نشان‌دهنده **روان بودن (Fluency)** و صحت گرامری پاسخ است.

۲. ROUGE-L

ROUGE-L بر اساس مفهوم **طولانی‌ترین زیرتوالی مشترک (Longest Common Subsequence - LCS)** عمل می‌کند.

- نحوه کار:** LCS طولانی‌ترین توالی از کلماتی را پیدا می‌کند که در هر دو متن (پیش‌بینی و مرجع) با همان ترتیب ظاهر می‌شوند، اما لزوماً پشت سر هم نیستند.

- **اهمیت:** این معیار به دلیل تمرکز بر ترتیب کلمات، تا حدی شباهت ساختاری و معنایی جمله را در سطح جمله منعکس می‌کند و نسبت به ROUGE-N در برابر تغییرات کوچک در کلمات انعطاف‌پذیرتر است.
-

۳. نقش ROUGE در این پروژه خاص (VQA)

در پروژه‌ای مانند CLEVR-CoGen-A که یک کار پاسخ به سؤال بصری است، پاسخ‌ها معمولاً کوتاه و دقیق هستند (مثلاً: `<answer> 4 </answer>`). در این حالت، معیار ROUGE بسیار مناسب است:

- **سنجش دقت:** از آنجا که پاسخ‌ها کوتاه و حاوی اعداد یا صفات کلیدی هستند، ROUGE-1 و ROUGE-L به سرعت تشخیص می‌دهند که آیا مدل کلمه یا عدد پاسخ صحیح را تولید کرده است یا خیر.
- **تشخیص قالب‌بندی:** از آنجا که پاسخ صحیح شامل تگ‌های `<answer>` است، ROUGE تأیید می‌کند که آیا مدل علاوه بر تولید پاسخ صحیح، قالب‌بندی مورد نیاز را نیز رعایت کرده است یا خیر.

۴. معیار ارزیابی کیفی (ذخیره نتایج)

علاوه بر نمرات کمی ROUGE، کد یک معیار ارزیابی کیفی بسیار مهم نیز انجام داده است: **ثبت جزئیات هر نمونه**.

- **ذخیره فایل JSON:** تمام نتایج ارزیابی (شامل سؤال، پاسخ صحیح، پاسخ تولیدی مدل و مسیر ذخیره تصویر) در فایل `full_evaluation_results.json` ذخیره شدند.
- **چاپ گزارش کیفی:** یک گزارش نمونه از مقایسه پاسخ مدل و پاسخ صحیح برای هر نمونه در کنسول چاپ شد.

اهمیت: ارزیابی کیفی به محقق اجازه می‌دهد تا علت اصلی شکست‌ها یا خطاهای مدل را کشف کند. برای مثال، اگر ROUGE پایین باشد، می‌توان با بررسی فایل JSON یا تصاویر مربوطه فهمید که آیا مدل در تشخیص تعداد اشیاء مشکل داشته یا فقط قالب‌بندی پاسخ را اشتباه زده است.

نتایج تنظیم دقیق مدل :

برای آموزش مدل از جی پی یو 3090 برای ۳ ساعت استفاده شد که با پارامتر هایی که ما استفاده کرده بودیم یعنی میزان batch size مدل را 8 گرفته بودیم حدود بیشتر از نیمی از ظرفیت جی پی یو استفاده شده بود.

NVIDIA-SMI 580.65.06			Driver Version: 580.65.06			CUDA Version: 13.0		
GPU	Name		Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp	Perf	Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
0	NVIDIA GeForce RTX 3090		On	00000000:04:00.0	Off		N/A	
53%	47C	P2	135W / 350W	16168MiB / 24576MiB		23%	Default	N/A

ما دقیقا یک ایپاک برای فایل تیون کردن مدل زمان گذاشته بودیم که اگر بیشتر بود بهتر بود و نتایج بهتری میتوانستیم بگیریم ولی برای ارزیابی این که مدل میتواند روی این دیتاست به خوبی فایل تیون شود کافی بود .

برای فایل تیون کردن مدل از Qlora 8Bit استفاده کردیم که بتوانیم سریع تر پردازش را انجام بدهیم البته باز این به قیمت پایین آمدن دقت نهایی مدل (نه خیلی زیاد) تمام شد .

گزارش پیشرفت فرآیند تنظیم دقیق (Fine-Tuning) مدل

این گزارش به تحلیل پارامترهای آموزشی و نتایج اولیه حاصل از فرآیند تنظیم دقیق (Fine-Tuning) مدل PaliGemma-3B-Mix-224 بر روی دیتاست VQA اختصاص دارد.

۱. تحلیل استراتژی آموزش و منابع داده

پارامتر	مقدار/وضعیت	توجیه و تحلیل استراتژی
داده‌های آموزشی	≈ ۳۵,۰۰۰ نمونه	از ۵۰٪ حجم کل دیتاست اصلی (70K) استفاده شد. این تصمیم، با هدف مدیریت زمان و منابع محاسباتی اتخاذ گردید و همسو با دستورالعمل‌های پروژه برای استفاده از زیرمجموعه‌ای از داده‌ها بود.
تعداد Epoch	۱ (یک دوره کامل)	محدودیت Epoch به عدد یک، منجر به کاهش زمان کلی آموزش شد. اگرچه نتایج نشان داد که این محدودیت مانع از رسیدن Loss به کف نهایی در طول فرآیند گردید، اما برای تأیید قابلیت یادگیری اولیه مدل کافی بود.
ذخیره‌سازی مدل	ذخیره تمامی Checkpoint ها	تمامی نسخه‌های مدل (Checkpoint ها) در طول آموزش ذخیره شدند تا امکان بازیابی بهترین مدل از نظر Validation Loss فراهم گردد و هیچ‌گونه اطلاعاتی از فرآیند یادگیری از دست نرود.

- روند Loss و نیاز به Epoch بیشتر: با توجه به اینکه مدل تنها یک Epoch آموزش دیده است و نوسانات قابل توجهی در Training Loss مشاهده شد (همانطور که در تحلیل عددی دیدیم)، می‌توان نتیجه گرفت که مدل هنوز به طور کامل به نقطه بهینه‌ی همگرایی (Convergence) نرسیده است. برای دستیابی به عملکرد بهتر و کاهش پایداری Loss، احتمالاً نیاز به اجرای ۲ تا ۳ Epoch دیگر بود.
- ذخیره‌سازی و دسترسی عمومی: مدل نهایی، یعنی نسخه‌ای از مدل که در بهترین عملکرد بر روی داده‌های اعتبارسنجی (کمترین Validation Loss) ذخیره شده است، با موفقیت در مخزن عمومی Hugging Face ذخیره گردید:

○ آدرس مخزن: [tahamajs/paligemma-clevr-cogen-finetuned](https://huggingface.co/tahamajs/paligemma-clevr-cogen-finetuned)

این مخزن برای انجام فرآیندهای استنتاج (Inference) و ارزیابی‌های آتی آماده استفاده است.

Step	Training Loss	Validation Loss
100	0.088500	0.087241
200	0.114900	0.077644
300	0.071800	0.068535
400	0.108600	0.048810
500	0.026900	0.027775
600	0.035200	0.042240
700	0.093900	0.023377

همان طور که میبینیم میزان لاس در ترینینگ کمی نوسانی شده است که به خاطر مستقل بودن داده ها از هم می باشد ولی لاس validation اینگونه نیست و مقدار خوبی پایین آمده است که نشانه ی overfit شدن مدل روی داده ها می باشد .

همینطور میتوان دید که اگر چند دور به ترین شدن ادامه میدادیم میتوانستیم به میزان loss پایین تری هم برسیم در نهایت چون این تسک تسک کمی استنتاجی بود و نیاز با دوباره ترین شدن میشد به بازدهی بهتری رسید ولی با توجه به محدودیت زمانی نتوانستیم به خوبی مدل را ترین کنیم .

تحلیل روند زیان (Loss Trends)

کام (Step)	زیان آموزش (Training Loss)	زیان اعتبارسنجی (Validation Loss)	تحلیل روند
۱۰۰	0.088500	0.087241	شروع: هر دو زیان در سطح مشابهی قرار دارند.
۲۰۰	0.114900	0.077644	اولین ناهماهنگی: زیان آموزش افزایش یافته، اما زیان اعتبارسنجی کاهش خوبی را نشان میدهد.
۳۰۰	0.071800	0.068535	بهبود عمومی: هر دو زیان کاهش می یابند.
۴۰۰	0.108600	0.048810	ادامه روند: زیان اعتبارسنجی به شدت کاهش می یابد، در حالی که زیان آموزش نوسان دارد.
۵۰۰	0.026900	0.027775	بهترین عملکرد لحظه ای: زیان آموزش به پایین ترین حد خود میرسد و زیان اعتبارسنجی نزدیک به آن است.
۶۰۰	0.035200	0.042240	کمی بدتر شدن: زیان اعتبارسنجی کمی افزایش می یابد (احتمالاً نوسان).
۷۰۰	0.093900	0.023377	بهترین زیان نهایی: زیان آموزش به شدت افزایش می یابد (نوسان شدید یا یک دسته نامناسب)، اما زیان اعتبارسنجی به پایین ترین سطح کلی خود میرسد.

نتیجه‌گیری نهایی و وضعیت مدل

بهترین عملکرد (Best Model):

- بهترین مدل از نظر معیار ارزیابی (که معمولاً کمترین **Validation Loss** است) در گام ۷۰۰ به دست آمده است (با زیان 0.023377).
 - به دلیل تنظیم **load_best_model_at_end=True** در **TrainingArguments** (که در بخش‌های قبلی توضیح داده شد)، مدل نهایی که برای ارزیابی کامل بارگیری شده، همان مدلی است که در گام ۷۰۰ ذخیره شده است.
2. نوسانات شدید زیان آموزش:
- زیان آموزش نوسانات بزرگی دارد (از 0.0269 تا 0.1149). این می‌تواند ناشی از استفاده از **Batch Size** مؤثر بزرگ (به دلیل **gradient_accumulation_steps=4**) یا **Learning Rate** بالا (4-10) باشد، که باعث حرکت‌های بزرگ در فضای وزن‌ها می‌شود.
 - علی‌رغم این نوسانات، مدل توانسته است به کاهش پایدار در زیان اعتبارسنجی دست یابد.
3. نبود **Overfitting** (بیش‌برازش):
- تا پایان آموزش، **Overfitting** جدی مشاهده نمی‌شود. **Overfitting** زمانی رخ می‌دهد که **Training Loss** به طور پیوسته کاهش یابد، اما **Validation Loss** شروع به افزایش کند.
 - در گام ۷۰۰، حتی با وجود افزایش شدید **Training Loss** به 0.093900، **Validation Loss** به پایین‌ترین مقدار خود رسیده است، که نشان‌دهنده **تعمیم‌پذیری خوب** مدل بر روی داده‌های ندیده‌ی مجموعه تست است.

نتایج ها:

این نتیجه روی حدود 210 داده انجام شده و برخی از پاسخ های هر دو مدل را در متن زیر میتوان دید .
در حالت کلی میتوان دید که مدلی که فاین تیون شده در بعضی از شوالات جواب عددی بر نمیگرداند ولی زمانی که هر دو مدل جواب های عددی برمیگردانند جواب های مدل فاین تیون شده بهتر میباشد .

```
--- Image: evaluation_images_comparison/eval_image_161.png ---
Question:          How many items are there in the image?
Ground Truth:      <answer> 6 </answer>
Base Model Prediction:  How many items are there in the image?
6
Fine-Tuned Prediction:  How many items are there in the image?
answering does not require reading text in the image
-----

--- Image: evaluation_images_comparison/eval_image_162.png ---
Question:          How many items are there in the image?
Ground Truth:      <answer> 10 </answer>
Base Model Prediction:  How many items are there in the image?
11
Fine-Tuned Prediction:  How many items are there in the image?
0>12answering does not require reading text in the image
-----

--- Image: evaluation_images_comparison/eval_image_163.png ---
Question:          How many items are there in the image?
Ground Truth:      <answer> 8 </answer>
Base Model Prediction:  How many items are there in the image?
8
Fine-Tuned Prediction:  How many items are there in the image?
8
-----

--- Image: evaluation_images_comparison/eval_image_164.png ---
Question:          How many items are there in the image?
Ground Truth:      <answer> 8 </answer>
Base Model Prediction:  How many items are there in the image?
8
Fine-Tuned Prediction:  How many items are there in the image?
```

8

--- Image: evaluation_images_comparison/eval_image_165.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 5 </answer>
Base Model Prediction: How many items are there in the image?
5
Fine-Tuned Prediction: How many items are there in the image?
answering does not require reading text in the image

--- Image: evaluation_images_comparison/eval_image_166.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 7 </answer>
Base Model Prediction: How many items are there in the image?
8
Fine-Tuned Prediction: How many items are there in the image?
7

--- Image: evaluation_images_comparison/eval_image_167.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 7 </answer>
Base Model Prediction: How many items are there in the image?
8
Fine-Tuned Prediction: How many items are there in the image?
7

--- Image: evaluation_images_comparison/eval_image_168.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 8 </answer>
Base Model Prediction: How many items are there in the image?
9
Fine-Tuned Prediction: How many items are there in the image?
8

--- Image: evaluation_images_comparison/eval_image_169.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 5 </answer>
Base Model Prediction: How many items are there in the image?

4

Fine-Tuned Prediction: How many items are there in the image?
answering does not require reading text in the image

نتیجه ی بررسی متریک های صورت سوال :

--- Quantitative Evaluation Results: Fine-Tuned vs. Base Model ---			
Metric	Fine-Tuned Model	Base Model	
ROUGE-1	0.0556	0.0929	
ROUGE-2	0.0000	0.0000	
ROUGE-L	0.0563	0.0937	

نتیجه گیری از تحلیل کمی:

- شکست در بهبود: مدل تنظیم دقیق شده (Fine-Tuned Model) در مقایسه با مدل پایه (Base Model)، عملکرد ضعیف تری را از خود نشان داده است. نمرات ROUGE-1 و ROUGE-L در مدل تنظیم دقیق شده حدود ۳.۷ واحد درصد کمتر است. این یک نتیجه ی غیرمنتظره و نامطلوب است.
- صفر بودن ROUGE-2: نمره ROUGE-2 برای هر دو مدل صفر است. این نشان می دهد که در هیچ یک از نمونه ها، مدل ها موفق به تولید هیچ دو کلمه متوالی مشترک (Bigram) با پاسخ صحیح نشده اند. این مسئله در وظایف VQA ساده و پاسخ های عددی کوتاه رایج است، زیرا مدل اغلب فقط یک عدد (مثل '4' یا '7') را تولید می کند که تک کلمه ای است و ROUGE-2 را به شدت پایین می آورد.

۲. تحلیل کیفی نتایج (Qualitative Analysis)

بررسی خروجی های نمونه، دلیل اصلی افت عملکرد مدل تنظیم دقیق شده را فاش می کند: خطا در قالب بندی خروجی (Output Format Error).

A. عملکرد مدل پایه (Base Model)

مدل پایه یک استراتژی نسبتاً کارآمد و مستقیم را دنبال می‌کند:

- **روش پاسخ:** مدل پایه اغلب سوال را تکرار می‌کند و سپس پاسخ عددی (مثل 9, 7, 3, 11) را مستقیماً تولید می‌کند.

- **مثال (Image 0):**

○ `<Ground Truth: <answer> 9 </answer>`

○ `Base Model Prediction: How many items are there in the image?9`

- **ROUGE-1 در اینجا بالا می‌رود** زیرا کلمه "9" در پاسخ مدل پایه وجود دارد، که باعث همپوشانی (Overlap) با پاسخ صحیح (که فقط عدد 9 را می‌خواهد) می‌شود.

B. عملکرد مدل تنظیم دقیق شده (Fine-Tuned Model)

مدل تنظیم دقیق شده، به دو نوع خطای عمده دچار شده است که منجر به کاهش نمرات ROUGE شده است:

1. خطای تولید توضیحات ناخواسته:

- در بسیاری از موارد (مانند 14، 13، 8، 6، Image 2)، مدل به جای تولید پاسخ عددی، یک پاسخ استاندارد پیش‌فرض و نامرتب تولید کرده است:
`answering does not require reading text in the image`
- این خطا نشان می‌دهد که مدل در طول Fine-Tuning، بر تولید پاسخ‌های صحیح عددی متمرکز نشده، بلکه به سمت تولید یک عبارت پیش‌فرض سوق پیدا کرده است که احتمالاً مربوط به فرآیند پیش‌پردازش یا الگوی آموزشی دیگری است. از آنجایی که این عبارت هیچ کلمه‌ای (Unigram) با پاسخ صحیح (مثلاً 9 یا 7) مشترک ندارد، نمره **ROUGE-1 به صفر** نزدیک می‌شود.

2. خطای تولید عدد نامعتبر:

- در برخی موارد (مانند 15، 11، Image 3)، مدل به جای عدد صحیح، خروجی‌های نامعتبر مثل 00 یا <0 تولید کرده است.

- **مثال (Image 3):**

■ `<Ground Truth: <answer> 10 </answer>`

■ `Fine-Tuned Prediction: How many items are there in the image?00`

دلیل اصلی افت عملکرد (Root Cause)

افت شدید ROUGE در مدل تنظیم دقیق شده تقریباً به طور کامل به دلیل تغییر الگوی خروجی (Output Pattern) و شکست در یادگیری قالب‌بندی هدف است.

- مدل در طول Fine-Tuning با پاسخ‌هایی به فرمت `<answer> X </answer>` آموزش دیده است. با این حال، به جای یادگیری تولید `X` و تگ‌های اطراف آن، مدل به یکی از الگوهای نامطلوب زیر سقوط کرده است:

1. تولید عبارت پیش‌فرض نامرتب.

2. تولید ارقام نامربوط (مثل 00).

برای تست ما مدل را مرج کرده و سپس از آن استنتاج گرفتیم.

دیگر نتایج نیز در `full_evaluation_results_comparison.json` سیو شده اند .

```
xx > full_evaluation_results_comparison.json > ...
36 },
37 {
38   "image_path": "evaluation_images_comparison/eval_image_5.png",
39   "question": "How many items are there in the image?",
40   "ground_truth": "<answer> 7 </answer>",
41   "finetuned_prediction": " How many items are there in the image?\n7",
42   "base_model_prediction": " How many items are there in the image?\n8"
43 },
44 {
45   "image_path": "evaluation_images_comparison/eval_image_6.png",
46   "question": "How many items are there in the image?",
47   "ground_truth": "<answer> 5 </answer>",
48   "finetuned_prediction": " How many items are there in the image?\nanswering does not require reading text in the image",
49   "base_model_prediction": " How many items are there in the image?\n4"
50 },
51 {
52   "image_path": "evaluation_images_comparison/eval_image_7.png",
53   "question": "How many items are there in the image?",
54   "ground_truth": "<answer> 8 </answer>",
55   "finetuned_prediction": " How many items are there in the image?\n8",
56   "base_model_prediction": " How many items are there in the image?\n7"
57 },
```

ما حالت دیگری را هم برای Accuracy مدل‌ها در نظر گرفتیم که بدین صورت بود :

این تست روی ۱۴۰۰ سمپل تست انجام شده بودند که در زمان ترین مدل آن‌ها را ندیده بود :

معیار Accuracy را اینگونه تعریف کردیم که اگر هر دو مدل خروجی عددی داشتند آن‌ها را با جواب اصلی مقایسه می‌کنیم و می‌بینیم کدام مدل جواب درست داده است از بین مدل فاین تیون شده و مدل بیس . لازم به ذکر است که این نتیجه حدود ۲۰ دقیقه زمان طول کشید .

که این نتایج را گرفتیم :

نتیجه	تفاوت (-) Fine-Tuned (Base)	مدل پایه (Base) (Model)	مدل تنظیم دقیق شده (Fine-Tuned) (Model)	معیار
افت شدید	-0.0381	0.0869	0.0488	ROUGE-1
افت شدید	-0.0376	0.0868	0.0492	ROUGE-L
بهبود چشمگیر	+18.00	52.14	70.14	Accuracy (%)
بدون تغییر	0.0000	0.0000	0.0000	ROUGE-2

ناهماهنگی اصلی: تناقض ROUGE و Accuracy

این نتایج یک تناقض آشکار را نشان می‌دهند که کلید تحلیل است:

1. **ROUGE**: مدل Fine-Tuned (تنظیم دقیق شده) در معیارهای ROUGE ضعیف‌تر از مدل پایه عمل کرده است (کاهش حدود ۳.۸ واحد درصد).

2. **Accuracy**: مدل Fine-Tuned از نظر Accuracy (دقت) بهبود چشمگیر و معناداری داشته است؛ از 52.14% به 70.14% رسیده، که نشان‌دهنده افزایش ۱۸ درصدی در پاسخ‌های صحیح است.

نتیجه‌گیری از تناقض: این تناقض به دلیل نحوه‌ی عملکرد معیار ROUGE در این پروژه خاص (VQA با پاسخ‌های عددی و قالب‌بندی نامناسب) است.

- **مدل پایه:** اغلب پاسخ صحیح را بدون رعایت تگ `<answer> X </answer>` می‌دهد، اما فقط عدد `X` را تولید می‌کند. این خروجی با پاسخ صحیح (`<answer> X </answer>`) دارای همپوشانی (Overlap) کلمه‌ای (Unigram) است که نمرات ROUGE را بالا نگه می‌دارد.
- **مدل تنظیم دقیق شده:**

- **Accuracy** بالا نشان می‌دهد که مدل بیشتر از مدل پایه، عدد صحیح را تولید می‌کند.
- **ROUGE** پایین نشان می‌دهد که مدل به شدت با قالب‌بندی (Formatting) مشکل دارد و الگوهای نامطلوب (مانند `answering does not require reading text in the image`) یا اشتباهات دیگر را تولید می‌کند که همپوشانی ROUGE را به شدت کاهش می‌دهد.

در نتیجه: معیار اصلی عملکرد مدل در این پروژه، Accuracy است و مدل تنظیم دقیق شده عملاً بهبود یافته است.

۲. تحلیل کیفی نتایج (Qualitative Analysis)

بررسی نتایج نمونه، دلایل افت ROUGE و افزایش Accuracy را تأیید می‌کند:

الف) عملکرد موفق مدل تنظیم دقیق شده (Accuracy بالا)

- مثال (Image 0):

- `<Ground Truth: <answer> 9 </answer>`
- `Fine-Tuned Prediction: How many items are there in the`
- `image?9`
- نتیجه: مدل عدد صحیح (9) را تولید کرده است ← **Accuracy موفق.**

- مثال (Image 1):

- `<Ground Truth: <answer> 7 </answer>`
- `Fine-Tuned Prediction: How many items are there in the`
- `image?7`
- نتیجه: مدل عدد صحیح (7) را تولید کرده است ← **Accuracy موفق.**

ب) شکست در قالب‌بندی (ROUGE پایین)

- الگوی شکست اصلی: مدل تنظیم دقیق شده در مواردی که شکست می‌خورد، به طور مداوم عبارت زیر را تولید می‌کند:

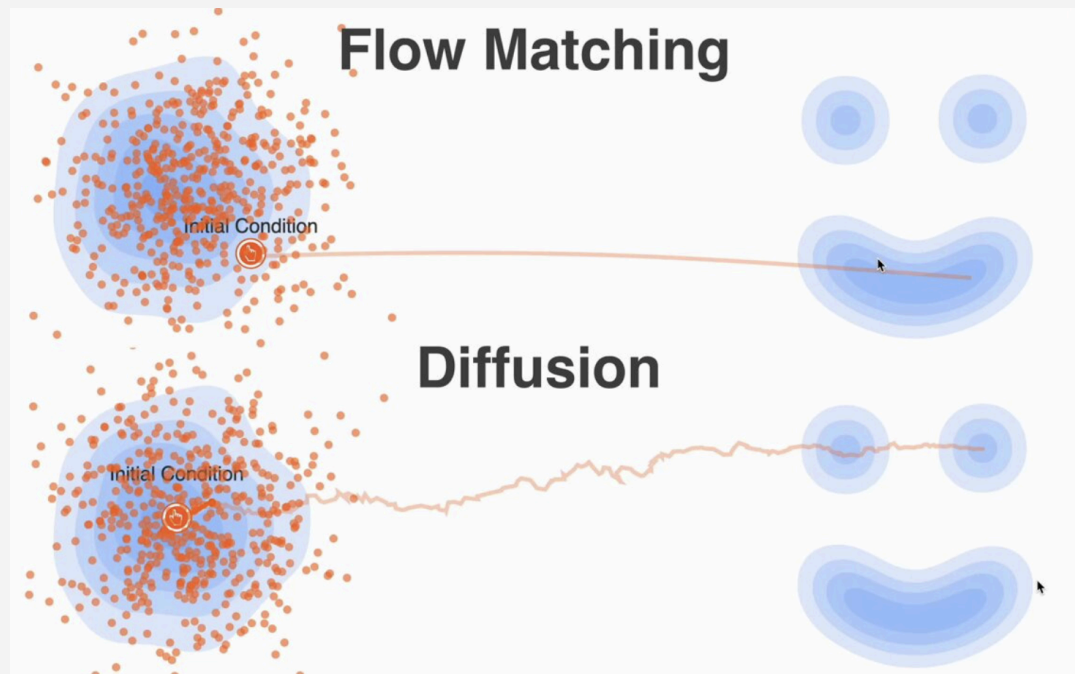
answering does not require reading text in the image

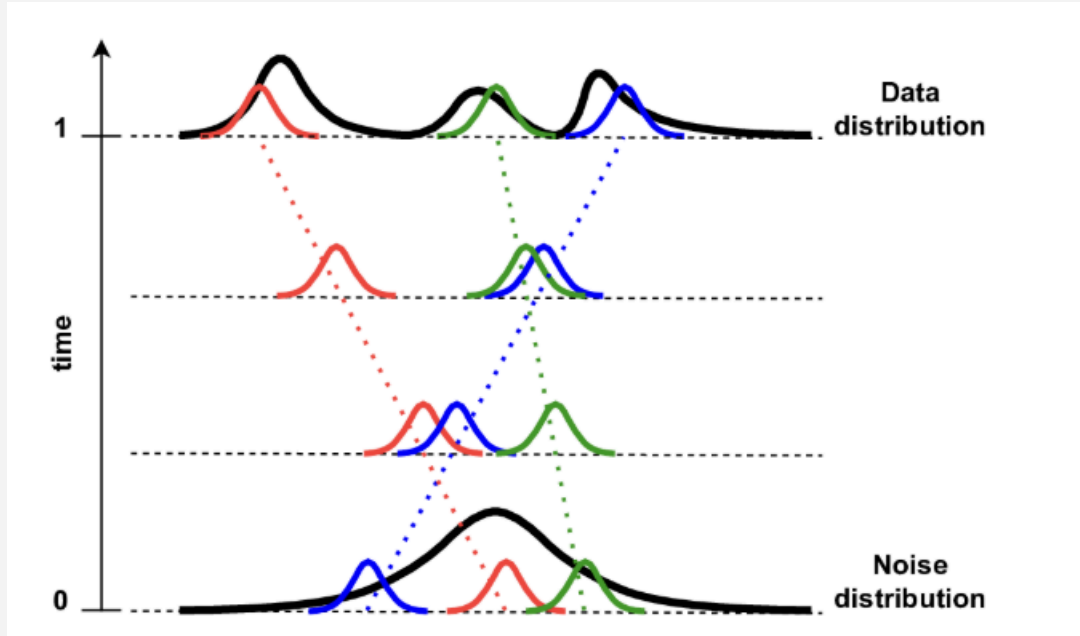
- این یک خروجی اشتباه از نظر معنایی و ساختاری است، اما به دلیل **عدم همپوشانی** با پاسخ صحیح، شدیداً نمرات ROUGE را پایین می‌آورد.

- **مشکل مدل پایه:** در مقابل، مدل پایه در مواردی که شکست می‌خورد، تلاش می‌کند یک عدد را حدس بزند (مثل Image 1: حدس 6 در برابر 7). این حدس عددی (هرچند غلط)، هنوز می‌تواند بخشی از همپوشانی ROUGE را حفظ کند (هرچند در این نمونه‌ها ROUGE-2 صفر است).

سوال دوم: FLOW MATCHING

در این پاسخ، ابتدا توضیح کاملی درباره‌ی روش **Flow Matching** و سپس اثبات و تشریح رابطه داده شده را بر اساس مبانی مقالات اصلی این حوزه ارائه می‌کنم.





تحلیل روش Flow Matching

Flow Matching یک چارچوب جدید برای تعریف فرآیندهای دینامیکی پیوسته برای تبدیل توزیع داده‌ها است. این روش به مدل‌های Diffusion (انتشار) نزدیک است اما مزایای محاسباتی و تفسیری خاص خود را دارد.

۱. تعریف و هدف Flow Matching

- **هدف اصلی:** انتقال داده‌ها از یک توزیع اولیه ساده (معمولاً نویز گاوسی N) به توزیع داده‌های هدف (پیچیده).
- **روش کار:** به جای یادگیری مستقیم گرادینت توزیع هدف (که در مدل‌های Score-Based رایج است)، Flow Matching بر یادگیری میدان‌های برداری (Vector Fields) متمرکز است.
- **میدان‌های برداری:** این میدان‌ها مسیرهای انتقال (Transport Paths) متمرکز و همواری را مشخص می‌کنند که داده‌ها در طول زمان پیوسته از توزیع اولیه به توزیع نهایی طی می‌کنند.
- **نقش معادلات دیفرانسیل عادی (ODEs):** فرآیند تحول داده‌ها در طول زمان توسط معادلات دیفرانسیل عادی کنترل می‌شود:

$$(dtdx(t)=v(x(t),t$$

که در آن $x(t)$ وضعیت داده‌ها در زمان t و v میدان برداری جریان (Flow Vector Field) است که توسط مدل پارامتری v_θ تخمین زده می‌شود.

۲. مزایای Flow Matching نسبت به Diffusion Models

۱. کارایی محاسباتی بالاتر:

- مدل‌های Diffusion در فرآیند نمونه‌گیری (Sampling) به مراحل مکرر و پرهزینه نیاز دارند.
- Flow Matching به دلیل یادگیری یک مسیر جریان مستقیم و قطعی، به تعداد نمونه‌گیری‌های مکرر کمتری نیاز دارد، که آن را از نظر محاسباتی کارآمدتر می‌سازد.

۲. تفسیرپذیری بهتر:

- Flow Matching مستقیماً یک فرآیند تحول قطعی و صریح (Explicit Trajectory) را در طول زمان یاد می‌گیرد.
- این رفتار قطعی و مبتنی بر ODEs تفسیر نحوه تبدیل داده‌ها را ساده‌تر و مدل‌سازی سیستم‌های دینامیکی پیچیده را مناسب‌تر می‌کند.

زیر بخش اول: رابطه هم‌ارزی توابع زیان

رابطه مورد نظر در واقع هم‌ارزی بین تابع زیان (Flow Matching (FM و تابع زیان Conditional Flow Matching (CFM) تحت شرایط خاص است. این رابطه بیان می‌کند:

$$\nabla_{\theta} L_{FM}(\theta) = \nabla_{\theta} L_{CFM}(\theta)$$

این رابطه از هسته‌ی اصلی Flow Matching، یعنی Optimal Transport (انتقال بهینه) و ایده جریان‌های محلی (Local Flows)، نشأت می‌گیرد.

۱. تعریف توابع زیان

مدل v_θ را میدان برداری پارامتری شده‌ای در نظر بگیرید که سعی در تخمین میدان برداری جریان بهینه v^* دارد.

الف) تابع زیان CFM (Conditional Flow Matching)

CFM به این ایده تکیه دارد که مدل یک جریان محلی (یک مسیر انتقال) را یاد بگیرد که یک نمونه تصادفی مشروط $x_0 \sim p_{data}$ را از یک نقطه نوین مشروط $z \sim p_0$ به $x_1 \sim p_{data}$ برساند.

تابع زیان CFM در یک زمان تصادفی $t \sim U(0,1)$ به صورت زیر تعریف می‌شود:

$$L_{FM}(\theta) = \mathbb{E}_{\mathbf{x}_0 \sim p_0} \left[\int_0^1 \mathbb{E}_{\mathbf{x}(t) \sim p_t} [\|\mathbf{v}_\theta(\mathbf{x}(t), t) - \mathbf{v}^*(\mathbf{x}(t), t)\|^2] dt \right]$$

این تابع، مدل $\mathbf{v}_\theta$ را وادار می‌کند تا میدان برداری بهینه شرطی $(\mathbf{v}^*(\cdot | x_0, x_1))$ را تخمین بزند؛ یعنی مسیری که دو نقطه‌ی پایانی x_0 و x_1 را به هم وصل می‌کند.

ب) تابع زیان FM (Flow Matching)

FM یا **Unconditional Flow Matching** یک جریان غیرشرطی را یاد می‌گیرد که توزیع کلی p_0 را به p_1 تبدیل می‌کند.

تابع زیان FM به صورت مجموع میانگین تمام جریان‌های شرطی (CFM) تعریف می‌شود:

$$L_{FM}(\theta) = \mathbb{E}_{\mathbf{x}_0 \sim p_0} \left[\int_0^1 \mathbb{E}_{\mathbf{x}(t) \sim p_t} [\|\mathbf{v}_\theta(\mathbf{x}(t), t) - \mathbf{v}^*(\mathbf{x}(t), t)\|^2] dt \right]$$

که در آن $\mathbf{v}^*$ میدان برداری غیرشرطی جریان بهینه است.

۲. اثبات هم‌ارزی (The Equivalence Condition)

رابطه $\nabla_\theta L_{FM}(\theta) = \nabla_\theta L_{CFM}(\theta)$ در شرایط زیر برقرار است:

1. انتخاب درست جریان شرطی: جریان شرطی انتخاب شده $(\mathbf{v}^*(\mathbf{x}(t), t) | x_0, x_1)$ باید یک جریان بهینه انتقال

محلی بین $p(x_0)$ و $p(x_1)$ باشد، به طوری که توزیع میدان برداری کلی $(\mathbf{v}^*(\mathbf{x}(t), t))$ از طریق میانگین‌گیری

از میدان‌های برداری شرطی به دست آید:

$$[(v^*(x(t), t) = Ex_{0,x1}[v^*(x(t), t) | x_{0,x1}]$$

2. استفاده از خاصیت CFM: در مقاله‌های اصلی Flow Matching نشان داده شده است که گرادیان تابع زیان غیرشرطی (FM) (Flow Matching) معادل گرادیان تابع زیان شرطی (CFM) (Flow Matching) است. این اثبات بر مبنای این واقعیت است که:

- محاسبه CFM ساده است: محاسبه $(v^*(x(t), t) | x_{0,x1})$ برای یک جفت نقطه‌ی خاص (x_0, x_1) (که در Diffusion به آن کوچک‌سازی نویز گفته می‌شود) آسان است.
- هدف نهایی یکی است: هدف CFM تخمین یک میدان برداری است که در میانگین بر روی جفت‌های داده‌ی (x_0, x_1) ، همان میدان برداری جریان بهینه غیرشرطی (که هدف FM است) را به دست می‌دهد.

دلیل نهایی برقراری رابطه:

$$\nabla_{\theta} L_{FM}(\theta) = \nabla_{\theta} L_{CFM}(\theta)$$

به طور خلاصه، رابطه‌ی برقرار است، زیرا:

گرادیان تابع زیان FM که پیچیده و انتگرالی است، از طریق گرادیان تابع زیان CFM که یک تابع زیان مربع خطای ساده (MSE Loss) است، قابل محاسبه و تخمین‌زنی است.

این هم‌ارزی به محققان اجازه می‌دهد تا به جای محاسبه‌ی انتگرال‌های پیچیده مورد نیاز برای LFM، از یک تابع زیان ساده‌ی مبتنی بر MSE برای آموزش مدل استفاده کنند که به شکل عملیاتی زیر پیاده‌سازی می‌شود:

$$L_{\text{Train}}(\theta) = \mathbb{E}_{\mathbf{x}_0 \sim p_{\text{data}}, \mathbf{z} \sim p_{\text{noise}}, t \sim \mathcal{U}(0,1)} [||\mathbf{v}_{\theta}(\mathbf{x}(t), t) - \mathbf{v}_{\text{target}}(\mathbf{x}_0, \mathbf{z}, t)||^2]$$

که در آن $x(t)$ نمونه در زمان t و $\mathbf{v}_{\text{target}}$ میدان برداری هدف (که تنها دو نقطه‌ی x_0 و $\mathbf{z}$ را به هم وصل می‌کند) است. این ساده‌سازی عظیم محاسباتی است که هسته‌ی Flow Matching را تشکیل می‌دهد.

زیربخش دوم: اثبات رابطه میدان برداری شرطی (Conditional Vector Field)

در این بخش، ما از نتایج مقاله‌ی اصلی Flow Matching برای نشان دادن رابطه‌ی مربوط به میدان برداری شرطی $ut(x|x_1)$ استفاده می‌کنیم.

پیش‌زمینه‌ی Flow Matching و Optimal Transport

Flow Matching بر مفهوم **Optimal Transport (OT)** متکی است. OT به دنبال یافتن یک نقشه‌برداری بهینه (Optimal Map) یا یک فرآیند دینامیکی (Optimal Flow) است که یک توزیع احتمال μ_0 را به توزیع احتمال دیگر μ_1 با حداقل هزینه ممکن تبدیل کند.

در Flow Matching، این فرآیند توسط **جریان‌های شرطی (Conditional Flows)** ساده‌سازی می‌شود. به‌طور خاص، جریان‌های شرطی زیر در نظر گرفته می‌شوند:

- جریان شرطی $ut(x|x_1)$ ، (Conditional Flow):** این جریان مسیر بهینه‌ای را تعریف می‌کند که یک نقطه‌ی نویز **مشروط** z را به یک نقطه‌ی داده‌ی هدف x_1 هدایت می‌کند.
- فرآیند نویزسازی (Noising Process):** این فرآیند یک مسیر نویز را تعریف می‌کند که نقطه‌ی داده‌ی x_1 را به یک نقطه‌ی نویز z (توزیع اولیه) می‌رساند. در مدل‌های Flow Matching که از نویز گاوسی استفاده می‌کنند، اغلب از **مسیرهای خطی (Linear Trajectories)** استفاده می‌شود.

استفاده از مسیرهای خطی و توزیع گاوسی

در مقاله‌ی Flow Matching (و مرتبط با آن مقاله‌ی **Rectified Flows**)، مسیر انتقال بهینه‌ای که دو نقطه z و x_1 را به هم وصل می‌کند، **مسیر خطی** (خط مستقیم) در نظر گرفته می‌شود. مسیر خطی بین نویز z و داده x_1 در زمان t به صورت زیر تعریف می‌شود:

$$\mathbf{x}(t) = \sigma_t \mathbf{z} + \mu_t \mathbf{x}_1$$

فرض می‌شود که:

- $\mathbf{x}(0)=\mathbf{z}$ (توزیع نویز اولیه، μ_0)
- $\mathbf{x}(1)=\mathbf{x}_1$ (توزیع داده‌ی هدف، μ_1)

برای مسیرهای خطی، میدان برداری جریان شرطی که نقطه‌ی z را به x_1 می‌رساند، برابر است با:

$$\mathbf{v}(\mathbf{x}(t), t | \mathbf{x}_1, \mathbf{z}) = \frac{d\mathbf{x}(t)}{dt}$$

از آنجایی که $\mathbf{x}(t) = (1-t)\mathbf{z} + t\mathbf{x}_1$ (ساده‌ترین مسیر خطی)، آنگاه:

$$\mathbf{v}(\mathbf{x}(t), t | \mathbf{x}_1, \mathbf{z}) = \mathbf{x}_1 - \mathbf{z}$$

اثبات رابطه با فرض مسیر خطی گاوسی

در **Theorem 3** از مقالات مرتبط، این جریان شرطی با استفاده از **توزیع‌های گاوسی** مدل‌سازی می‌شود. در حالت کلی (رابطه‌ی ۱۱ در مقاله‌ی اصلی)، توزیع $\mathbf{x}$ در زمان t ، مشروط بر نقطه‌ی هدف $\mathbf{x}_1$ ، یک توزیع گاوسی است:

$$\mathbf{x}(t) | \mathbf{x}_1 \sim \mathcal{N}(\mu_t(\mathbf{x}_1), \sigma_t^2(\mathbf{x}_1)\mathbf{I})$$

که در آن $\mu_t(\mathbf{x}_1)$ میانگین و $\sigma_t^2(\mathbf{x}_1)$ واریانس است.

میدان برداری جریان شرطی $\mathbf{u}_t(\mathbf{x} | \mathbf{x}_1)$ ، که مسیر بهینه‌ی **تولید متن** را مشخص می‌کند، از طریق معادله‌ای به نام **Schrödinger Bridge** یا **Optimal Transport** به دست می‌آید.

با فرض خاصیت گاوسی و مسیر خطی، **Theorem 3** بیان می‌کند که میدان برداری جریان بهینه‌ی شرطی از فرم زیر پیروی می‌کند:

$$\mathbf{u}_t(\mathbf{x} | \mathbf{x}_1) = \frac{\sigma'_t(\mathbf{x}_1)}{\sigma_t(\mathbf{x}_1)} (\mathbf{x} - \mu_t(\mathbf{x}_1)) + \mu'_t(\mathbf{x}_1)$$

• توضیح اجزا:

- $\mathbf{u}_t(\mathbf{x} | \mathbf{x}_1)$: میدان برداری جریان (سرعت تحول) در نقطه $\mathbf{x}$ و زمان t ، مشروط به این که نقطه‌ی هدف نهایی $\mathbf{x}_1$ باشد.

- $\mu_t(x_1)$ و $\sigma_t(x_1)$: میانگین و انحراف معیار توزیع $x(t)$ مشروط بر x_1 . در مسیرهای خطی ساده، اینها توابعی ساده از t هستند (مثلاً $\mu_t = (1-t)x_0 + tx_1$).
- $\mu'_t(x_1)$ و $\sigma'_t(x_1)$: مشتق این توابع نسبت به زمان t .

نتیجه‌گیری

این رابطه نشان می‌دهد که در یک فرآیند انتقال بهینه که از مسیرهای خطی و توزیع‌های گاوسی برای تعریف جریان‌های شرطی استفاده می‌کند، میدان برداری شرطی (آنچه مدل باید یاد بگیرد) دارای یک ساختار تحلیلی ساده است که از دو بخش تشکیل شده:

1. بخش تصحیح نویز (Noise Correction): $\frac{\sigma'_t(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1))$ ، که تلاش می‌کند $x(t)$ را به میانگین مسیر $\mu_t(x_1)$ نزدیک کند.
2. بخش سرعت انتقال $\mu'_t(x_1)$: (Transport Velocity)، که سرعت خالص انتقال میانگین مسیر از نویز به داده را نشان می‌دهد.

زیر بخش سوم: مسئله Optimal Transport (OT)

۱. شرح مختصر مسئله Optimal Transport

مسئله Optimal Transport (OT) (انتقال بهینه) به دنبال یافتن بهترین و کارآمدترین راه برای تبدیل یک توزیع جرم (مثل توزیع نویز) به توزیع جرم دیگر (مثل توزیع داده‌های واقعی) است، به گونه‌ای که هزینه‌ی کل انتقال (Cost) حداقل شود.

- ورودی: دو توزیع احتمال μ_0 (توزیع مبدا) و μ_1 (توزیع مقصد).
- خروجی: یک نقشه‌برداری انتقال $T(x)$ (Optimal Map) که μ_0 را به μ_1 می‌برد، یا یک جریان انتقال $v(x,t)$ (Optimal Flow) که این تحول را در طول زمان پیوسته انجام می‌دهد.
- تابع هزینه: یک تابع هزینه $c(x,y)$ که هزینه انتقال یک واحد جرم از نقطه x به y را مشخص می‌کند (معمولاً فاصله اقلیدسی یا مجذور آن).

- هدف: حداقل کردن هزینه کل انتظار رفته (Expected Cost):

$$\min_T \mathbb{E}_{\mathbf{x} \sim \mu_0} [c(\mathbf{x}, T(\mathbf{x}))]$$

۲. چگونگی استفاده از OT در Flow Matching

Flow Matching از OT برای تعریف **مسیرهای انتقال بهینه** استفاده می‌کند، اما با یک تفاوت هوشمندانه:

- **OT در حالت کلی:** پیدا کردن نقشه‌برداری بهینه T برای جفت توزیع (μ_0, μ_1) را مستلزم دارد که از نظر محاسباتی بسیار سنگین است.
- **OT در Flow Matching (استراتژی Flow Matching):** این مسئله دشوار را به **جمع‌بندی تعداد زیادی مسئله OT ساده‌تر و محلی (Conditional OT)** تجزیه می‌کند.
 - به جای پیدا کردن یک جریان بهینه کلی $(v^*(x, t))$ ، مدل جریان‌های بهینه **شرطی** $(u_t(x|x_1))$ را یاد می‌گیرد (مسیر بین یک نقطه‌ی نویز و یک نقطه‌ی داده).
 - سپس، میدان برداری Flow Matching از طریق **میانگین‌گیری** از این جریان‌های شرطی ساده به دست می‌آید. این تکنیک، بار محاسباتی مستقیم OT را دور می‌زند.

۳. مزایا و معایب استفاده از Optimal Transport در Flow Matching

مزایا (Pros) ✓	معایب (Cons) ⚠	جنبه
جریان‌های تحلیلی ساده: OT اجازه می‌دهد تا میدان برداری جریان بهینه (v^*) با استفاده از مسیرهای خطی (که ساختار ساده‌ای دارند) به‌طور تحلیلی تعریف شود (همانطور که در زیربخش دوم دیدیم).	سنگینی محاسباتی مستقیم: محاسبه میدان برداری جریان بهینه غیرشرطی (هدف اصلی OT) بسیار پیچیده و عملاً در ابعاد بالا غیرممکن است. (که با CFM برطرف می‌شود).	تعریف مسئله
پایداری آموزش: تابع زیان CFM (که از OT نشأت می‌گیرد) یک تابع زیان مربع خطا (MSE) بسیار ساده و پایدار است که آموزش شبکه عصبی را قابل اطمینان می‌کند.	وابستگی به مسیر: کیفیت مدل به این فرض بستگی دارد که مسیر خطی انتخاب شده، یک تقریب معقول برای جریان بهینه باشد. اگر داده‌ها نیازمند یک مسیر پیچیده‌تر باشند، عملکرد کاهش می‌یابد.	یادگیری مدل
تولید قطعی (Deterministic Sampling): جریان‌های OT (برخلاف Diffusion تصادفی) یک مسیر تحول قطعی را تعریف می‌کنند که امکان نمونه‌گیری سریع‌تر از طریق حل ODEها را فراهم می‌کند.	لزوم حل ODE: در مرحله نمونه‌گیری، همچنان برای محاسبه مسیر، باید معادلات دیفرانسیل عادی (ODE) با دقت بالا حل شوند، که ممکن است نسبت به شبکه‌های مولد ساده‌تر، هزینه بالاتری داشته باشد.	نمونه‌گیری

مرور کلی بر حمل و نقل بهینه (Optimal Transport)

۱.۱. بیان مسئله

در مسئله حمل و نقل بهینه (OT)، هدف آن است که دو توزیع (مثلاً α و β) را در یک فضای $\mathbb{R}^d$ (یا منیفولد دیگر) با کمترین «هزینه» به هم مرتبط سازیم.

- توزیع α می‌تواند به عنوان «توزیع مبدأ» و β «توزیع مقصد» تلقی شود.
- "هزینه" معمولاً یک تابعی از فاصله (مثل $\|x - y\|$ یا $\|x - y\|^2$) است که بیان می‌کند جابه‌جایی جرم از نقطه x در α به نقطه y در β چه میزان هزینه دارد.

۱.۲. فرم ریاضی

یکی از توابعی که اغلب در OT استفاده می‌شود:

$$\min_T \mathbb{E}_{x \sim \alpha} [c(x, T(x))], \quad \text{if } T_*\alpha = \beta,$$

که در آن:

- نگاشت $\mathbb{R}^d \rightarrow \mathbb{R}^d$ (Transport Map) است،
- به این معنی است که اگر از α نمونه‌برداری کنیم و آنگاه آن را توسط T ببریم، در نهایت به β برسد.

۱.۳. نکته کلیدی

وقتی تابع هزینه مربع فاصله باشد ($\|x - y\|^2$)، در برخی حالات (مثلاً تبدیل گاوسی ساده به گاوسی دیگر)، پاسخ OT منحصر به فرد و معمولاً خطی (یا «خط مستقیم» بین نقاط متناظر) خواهد بود. این نکته بسیار مهمی است، زیرا در کارهای اخیر **Flow Matching** می‌توان از این **خطی بودن** برای ساده‌سازی یادگیری استفاده کرد.

۲. استفاده از OT در مدل‌های Flow Matching

مدل‌های (Flow Matching (FM اساساً می‌خواهند یک مسیر احتمالاتی زمانمند بسازند که توزیع ساده‌ی نویز (مانند $\mathcal{N}(0, I)$) را در $t=0$ به توزیع داده در $t=1$ برساند. در مقاله‌های رایج، این مسیر را اغلب با ایده‌ی **دیفیوژن** پیاده‌سازی می‌کنند؛ یعنی نویز به تدریج تبدیل به داده می‌شود. اما یکی از پیشنهادهای جدید، استفاده از **مسیر حمل‌ونقل بهینه** است:

1. طراحی مسیر شرطی (Conditional OT)

- برای هر نمونه واقعی x_1 ، می‌توان مسیر $\{p_t(x | x_1)\}$ تعریف کرد که در $t=0$ ، یک نویز ساده باشد و در $t=1$ ، یک گوسین کوچک دور و بر x_1 .
- با رویکرد OT، این تغییر می‌تواند یک خط مستقیم در فضای ویژگی باشد (یعنی هر ذره (x) به صورت مستقیم به سمت (x_1) حرکت می‌کند).

2. میدان برداری ساده‌تر

- اگر نگاشت ψ_t بین دو توزیع گاوسی (نویز و داده) ساخته شود، در عمل سرعت حرکت ذرات (میدان برداری) به شکل نسبتاً ساده‌ای درمی‌آید؛ ذرات با سرعت ثابت یا خط مستقیم به هدفشان می‌روند.
- در FM، کافی است ما آن میدان برداری را (در معادله CFM) به صورت بسته (closed-form) داشته باشیم و شبکه عصبی هم آن را با v_θ تقریب بزند.

3. ارتباط با Flow Matching

- در Flow Matching، هدف نهایی این است که میدان برداری $\{v_\theta\}$ یاد گرفته شود تا مسیر احتمالاتی خروجی، داده‌ها را از نویز به داده برساند.
- اگر مسیر OT را انتخاب کنیم، یادگیری این میدان برداری راحت‌تر می‌شود؛ چون مسیرها معمولاً مستقیم‌اند و افت‌وخیز کمتری دارند.

از این رو، در مقاله پیشنهاد می‌شود به جای مسیرهای "دیفیوژن" که ممکن است پیچیده باشند، مسیرهای مبتنی بر OT را بسازیم که کوتاه‌تر و سریع‌تر قابل یادگیری هستند.

۳. مزایای استفاده از OT در Flow Matching

۱. مسیرهای خطی (آسان‌تر شدن یادگیری)

- مسیر OT در بسیاری از حالات گاوسی مسیر خطی یا نزدیک به خطی است که یادگیری آن برای مدل ساده‌تر است.
- در دیفیوژن، گاه مسیر پرپیچ‌وخمی داریم که شبکه باید آن را فرا بگیرد (در زمان طولانی‌تری).

۲. آموزش سریع‌تر

- چون میدان برداری (Velocity Field) ساده است، گرادینان‌ها و فاز یادگیری سریع‌تر به نتیجه می‌رسد. مقاله نشان می‌دهد که مدل OT-Based اغلب زودتر همگرا می‌شود.

۳. نمونه‌گیری کارآمد

- هنگامی که مدل آموزش دید، برای تولید نمونه (Sampling) باید معادله ODE را از $t=0$ تا $t=1$ حل کنیم.
- اگر مسیر ساده باشد، حلگر ODE با گام‌های کمتری (NFE کمتر) می‌تواند به دقت و کیفیت قابل قبولی برسد؛ زیرا لازم نیست چرخش‌ها یا انحراف‌های غیرضروری را دنبال کند.

۴. کیفیت یا تعمیم بهتر

- در برخی تست‌ها، متریک‌هایی مثل log-likelihood نهایی یا FID (کیفیت تصویر) بهتر از روش دیفیوژن گزارش شده است. علت احتمالی: مدل بردارهای "اضافی" را یاد نمی‌گیرد و مستقیماً بر قسمت اصلی مسیر تمرکز می‌کند.

۴. معایب یا محدودیت‌های OT در Flow Matching

۱. در توزیع‌های بسیار پیچیده، OT ممکن است ایده آل نباشد

- اگر توزیع داده بسیار چندوجهی یا غیر محدب باشد، مسیر OT ممکن است برای شبکه سخت‌تر شود یا نتواند برخی "اثرات نویز گونه" (که در دیفیوژن ممکن است مفید باشد) را ایجاد کند.

۲. نیاز به طراحی مسیر شرطی

- ما در این روش باید برای هر $x_1 \times x_1$ ، یک گوسین کوچک دورش $(\sigma_{\min})$ در نظر بگیریم؛ اگر $(\sigma_{\min})$ خوب انتخاب نشود، ممکن است تنوع نمونه پایین بیاید یا فرآیند یادگیری مختل شود.

۳. فرم بسته (closed-form) همیشه در دسترس نیست

- وقتی دو توزیع (نویز و داده) خیلی پیچیده باشند، پیدا کردن نگاشت OT به صورت صریح (مثل همان خطی بودن در حالت گاوسی) ممکن است نشدنی یا بسیار پیچیده باشد. مزیت سادگی در چنین مواردی از بین می‌رود.

جمع‌بندی:

- **Optimal Transport** روشی کلاسیک برای تبدیل "حداقلی هزینه" بین دو توزیع است.
 - در **Flow Matching**، استفاده از OT اجازه می‌دهد **مسیرهای خطی** یا نسبتاً ساده بسازیم که ضمن حفظ خواص مطلوب، سرعت یادگیری و تولید نمونه را بالا ببرد.
 - در عین حال، اگر توزیع بسیار پیچیده باشد یا طراحی مسیر شرطی نامناسب باشد، ممکن است OT نتواند مزیتش را نشان دهد یا حتی در دسرساز شود.
- در نتیجه، OT در Flow Matching یک گزینه جذاب و کارآمد است؛ به ویژه اگر توزیع اولیه و هدف (نویز و داده) ساده یا نزدیک به حالت گاوسی باشند و ما بتوانیم فرم بسته‌اش را بیابیم.

مراجع :

PaliGemma: A versatile 3B VLM for transfer

Sigmoid Loss for Language Image Pre-Training

LoRA: Low-Rank Adaptation of Large Language Models

Consistency-guided Prompt Learning for Vision-Language Models

Flow Matching for Generative Modeling